Week 8 - Multivariate Analysis

Please run the cells of the notebook as you get to them while reading

In [1]:
```
import pandas as pd
import numpy as np
from datetime import datetime, timedelta
import seaborn as sns
from sklearn import decomposition
from sklearn import linear_model
from sklearn import metrics
```

1. Lesson: Multivariate graphs

In this lesson, we'll consider some multivariate plots: heatmaps and bubble plots. We'll also consider principal component analysis and multiple regression.

Heatmaps

In this plot, we use a heatmap to count the total traffic tickets given to citizens. A heatmap requires the x and y axes to be categorical, while the color involves a third, numerical variable. In this example, each row of the DataFrame represents a traffic ticket given to one person in a particular city and year. The data are given as three columns (city, year, and ticket), so we must use a pivot_table to transform it into a two-dimensional grid. The cmap is set to "Blues" in the first example, meaning that the heatmap uses only blue as its color. However, we could also use a multicolor map like "viridis" in the second example. I personally think "Blues" is easier to read - what do you think? Here is a reference guide to the color maps:

https://matplotlib.org/stable/gallery/color/colormap_reference.html

In [2]:
```
np.random.seed(0)
num_samples = 100
city_series = np.random.choice(["New York", "Boston", "Atlanta", "Wash
year_series = np.random.choice([2018, 2019, 2020], size = num_samples)
ticket_series = np.random.randint(50, 150, size = num_samples) # Traff
df = pd.DataFrame({"city": city_series, "year": year_series, "ticket":
pivot = df.pivot_table(index = ["city"], columns = ["year"], values =
```

In [3]:
```
df.iloc[0:5] # first five rows of a long DataFrame
```

Out[3]:

	city	year	ticket
0	New York	2019	79
1	Washington, D. C.	2020	71
2	Boston	2018	75
3	New York	2018	130
4	Washington, D. C.	2020	110

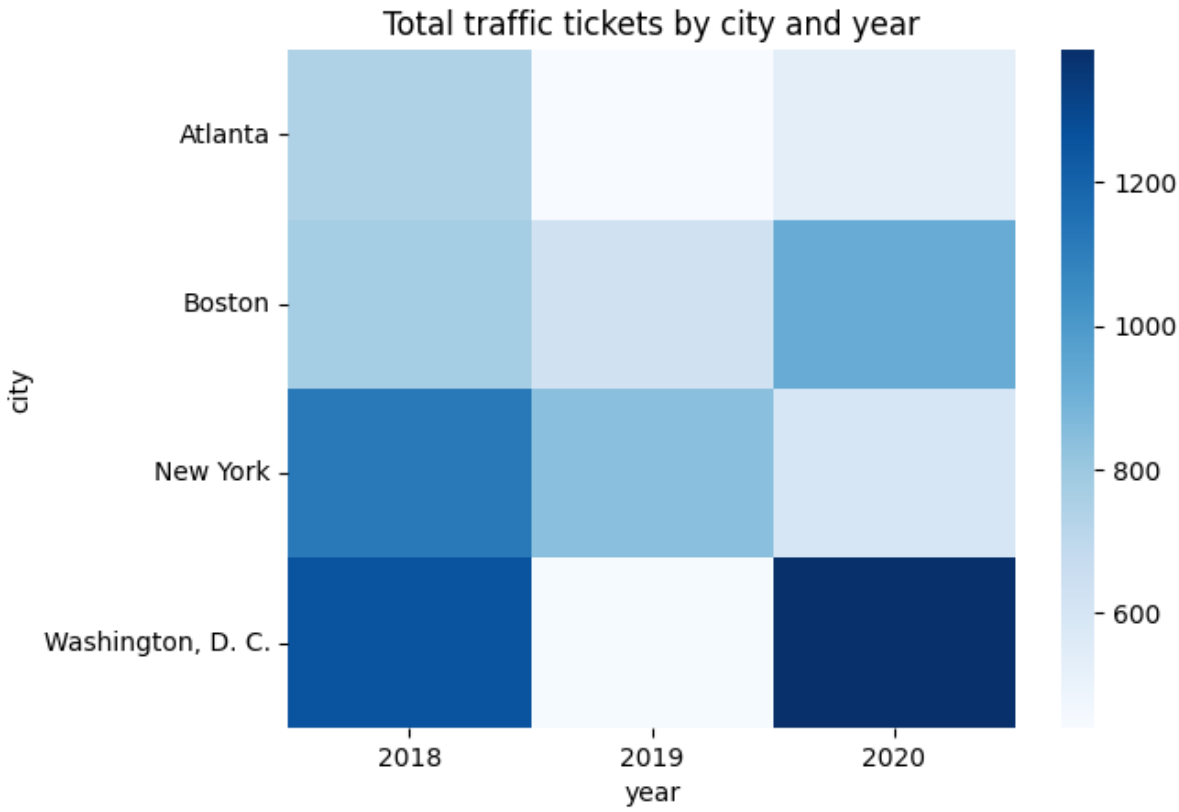
In [4]:
```
pivot # 2 by 2 grid showing total ticket amounts
```

Out[4]:

year	2018	2019	2020
city			
Atlanta	742	440	538
Boston	775	628	922
New York	1122	841	596
Washington, D. C.	1258	451	1385

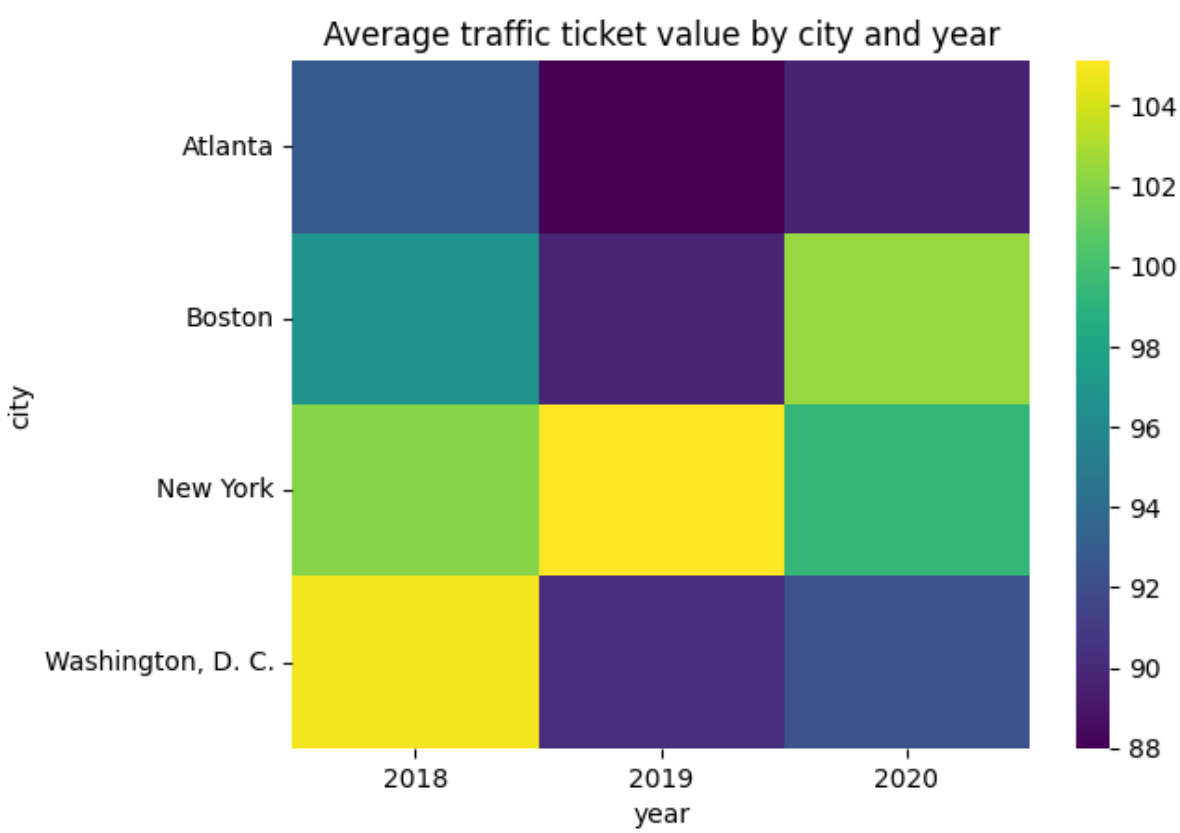
In [5]:
```
ax = sns.heatmap(pivot, cmap = "Blues")
ax.set_title("Total traffic tickets by city and year")
```

Out[5]:
```
Text(0.5, 1.0, 'Total traffic tickets by city and year')
```



In [6]:
```
pivot2 = df.pivot_table(index = ["city"], columns = ["year"], values =
ax2 = sns.heatmap(pivot2, cmap = "viridis") # do you think this is a b
ax2.set_title("Average traffic ticket value by city and year")
```

Out[6]: Text(0.5, 1.0, 'Average traffic ticket value by city and year')

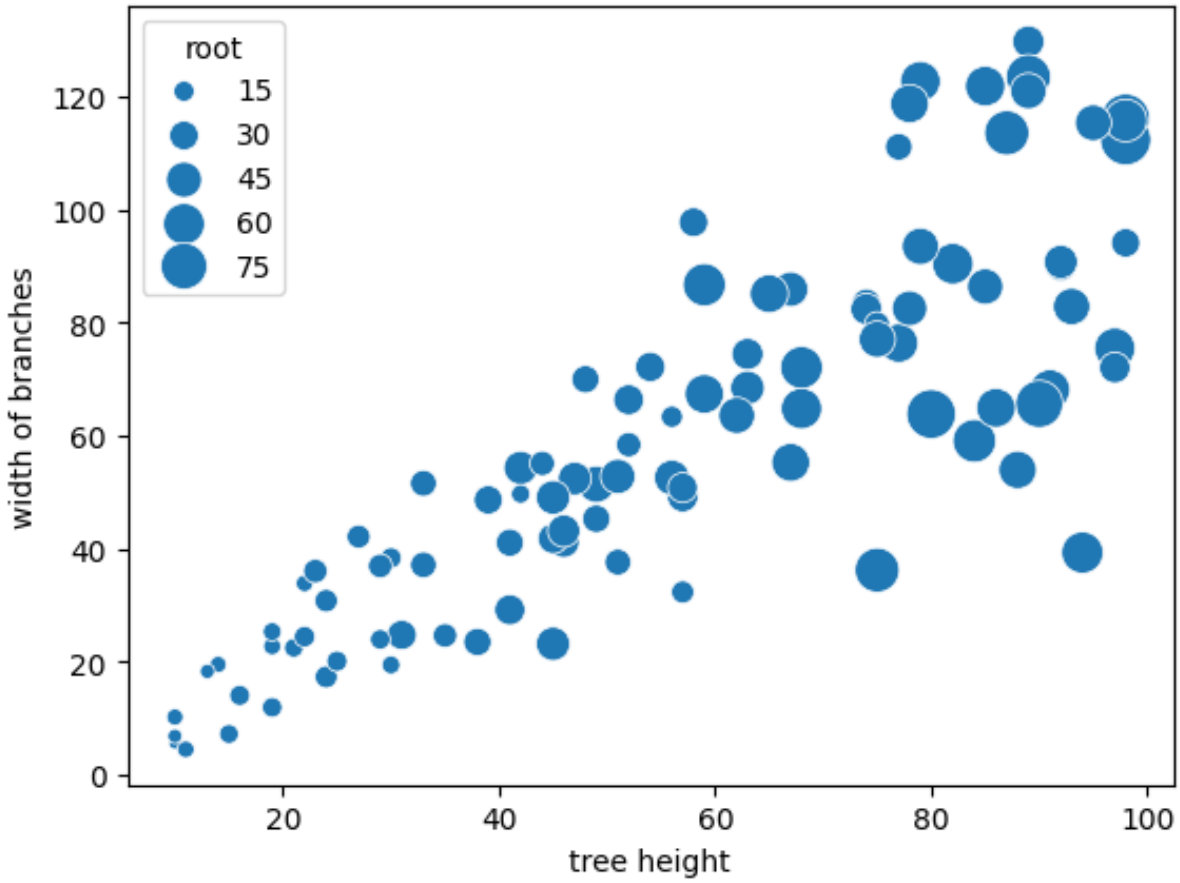


Bubble plots

A bubble plot is just a scatter plot, but with the dot size representing a third variable. Here, each data point shows a tree (I mean the kind of tree that is a plant and grows in the ground, not a decision tree), with its height, width of branches, and root depth. The bubble size in the bubble plot represents the root depth, while the other variables are shown on the x and y axes. From this plot, we can see that the root depth is strongly correlated with both the width of branches and the tree height.

In [7]:
```
np.random.seed(0)
num_samples = 100
height_series = np.random.randint(10, 100, size = num_samples) # tree
width_series = height_series * np.maximum(np.random.normal(scale = 0.3
root_series = height_series * np.maximum(np.random.normal(scale = 0.2,
df2 = pd.DataFrame({'height': height_series, 'width': width_series, 'r
minroot = root_series.min()
maxroot = root_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df2, x = "height", y = "width", size = "ro
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
```

Out[7]:
```
Text(0, 0.5, 'width of branches')
```



Principal Component Analysis

Principal Component Analysis can be thought of as a way to use one (or a few) linear combinations of the coordinates in order to explain all of the coordinates. It is most useful when there are strong correlations between some or all of the coordinates. Principal Component Analysis can show us which linear combinations are most (and least) useful for explaining the data.

With the trees data, the three values (height, width, and root) are highly correlated. For this reason, Principal Component Analysis shows that most of the variance can be explained by a single value which is a linear combination of height, width, and root. What this means is that if the first components_ vector in the code below (the "first principal component") is [a, b, c], for some specific numbers a, b, and c, then the value v = a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r), where the mu's are the means, is highly informative about the values of height, width, and root. In this case, because height, width, and root are so highly correlated, we can almost assume that all of the data is located along an axis defined by the vector [a, b, c]. The point v * [a, b, c] + (mean vector) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (mean vector) is therefore approximately equal to the data point (height, width, root) in

question. The mean vector is [mu_h, mu_w, mu_r]. Importantly, there is still variance unexplained by this one value - just knowing v * [a, b, c] does not tell you the height, width, and root with 100% accuracy. If the second principal component is [d, e, f], then we could approximate the tree vector even more accurately as: (height, width, root) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (d * (height - mu_h) + e * (width - mu_w) + f * (root - mu_r)) * [d, e, f] + (mean vector).

The printing of "components_", below, shows the first principal component [a, b, c]. If you draw a vector in the direction shown, you will be heading in the direction where the values height, width, and root change most rapidly. A later computation shows all three principal component vectors.

Optional exercise for the reader: show that if all three principal components are included, then equality is exact. This is going to be true simply because the principal components are orthonormal: the projections onto a complete set of orthonormal vectors always add up to the original vector.

(height, width, root) = (a * (height - mu_h) + b * (width - mu_w) + c * (root - mu_r)) * [a, b, c] + (d * (height - mu_h) + e * (width - mu_w) + f * (root - mu_r)) * [d, e, f] + (g * (height - mu_h) + h * (width - mu_w) + i * (root - mu_r)) * [g, h, i] + [mu_h, mu_w, mu_r].

For this proof, we must use the fact that the principal component vectors are orthonormal. That is, any pair of them are orthogonal, even with a transpose (ad + be + cf = 0, ag + bh + ci = 0, dg + eh + fi = 0, ab + de + gh = 0, ac + df + gi = 0, bc + ef + hi = 0) and any one of them is a unit vector, even with a transpose (a^2 + b^c + c^2 = 1, d^2 + e^2 + f^2 = 1, g^2 + h^2 + i^2 = 1, a^2 + d^2 + g^2 = 1, b^2 + e^2 + h^2 = 1, c^2 + f^2 + i^2 = 1). If you write out the equation above as a single vector, then apply the orthonormality rules, you should be able to prove equality.

In [8]:
```
pca = decomposition.PCA(n_components = 1)
pca.fit(df2)
print(f"The first principal component explains {pca.explained_variance
print(f"The components_ vector is [a, b, c] = {pca.components_}")
mu_h, mu_w, mu_r = df2.mean().values
print(f"(height, width, root) is approximately equal to ({pca.componen
print("Examples: (estimating the the vector equals its projection onto
df_example = df2.copy()
df_example["v"] = pca.components_[0][0] * (df2["height"] - mu_h) + pca
df_example["height_est"] = df_example["v"] * pca.components_[0][0] + m
df_example["width_est"] = df_example["v"] * pca.components_[0][1] + mu
df_example["root_est"] = df_example["v"] * pca.components_[0][2] + mu_
print(df_example.iloc[0:5])
```

```
print("Means:")
print(f"mu_h = {mu_h:.3}", f"mu_w = {mu_w:.3}", f"mu_r = {mu_r:.3}")
```

```
The first principal component explains 0.8686108480944689 of the varian
ce
The components_ vector is [a, b, c] = [[0.59031629 0.71908728 0.3666608
3]]
(height, width, root) is approximately equal to (0.59 * (height - 5.6e+
01) + 0.72 * (width - 5.9e+01) + 0.37 * (root - 3.6e+01)) * (0.59, 0.7
2, 0.37) + (5.6e+01, 5.9e+01, 3.6e+01)
Examples: (estimating the the vector equals its projection onto the fir
st principal component)
   height       width       root          v  height_est  width_est   ro
ot_est
0      54   72.139334  31.517932   6.287751   60.001762  63.563941  38.
673095
1      57   49.169527  33.768693  -7.633330   51.783921  53.553468  33.
568780
2      74   83.443733  28.770850  25.215680   71.175227  77.174773  45.
613225
3      77   76.353866  53.061241  30.794730   74.468631  81.186597  47.
658844
4      77  111.095365  26.006307  45.856916   83.360084  92.017624  53.
181557
Means:
mu_h = 56.3 mu_w = 59.0 mu_r = 36.4
```

If we were to choose a second or third component to preserve, that values below show that each would explain a small, additional amount of the variance. This means that with this particular dataset, we should likely do PCA using only one component and not two. If the components' explained variances were closer in magnitude to each other, we might have to include two or even all three values to explain the data.

In [9]:
```
pca3 = decomposition.PCA(n_components = 3)
pca3.fit(df2)
print(pca3.explained_variance_ratio_)
print(pca3.components_)
```

```
[0.86861085 0.09414617 0.03724298]
[[ 0.59031629  0.71908728  0.36666083]
 [ 0.32633701 -0.62809127  0.70640322]
 [ 0.73826204 -0.29734633 -0.6054373 ]]
```

The principal component vectors shown are orthogonal to each other (their dot product is basically zero). Here, we check that using the first two vectors.

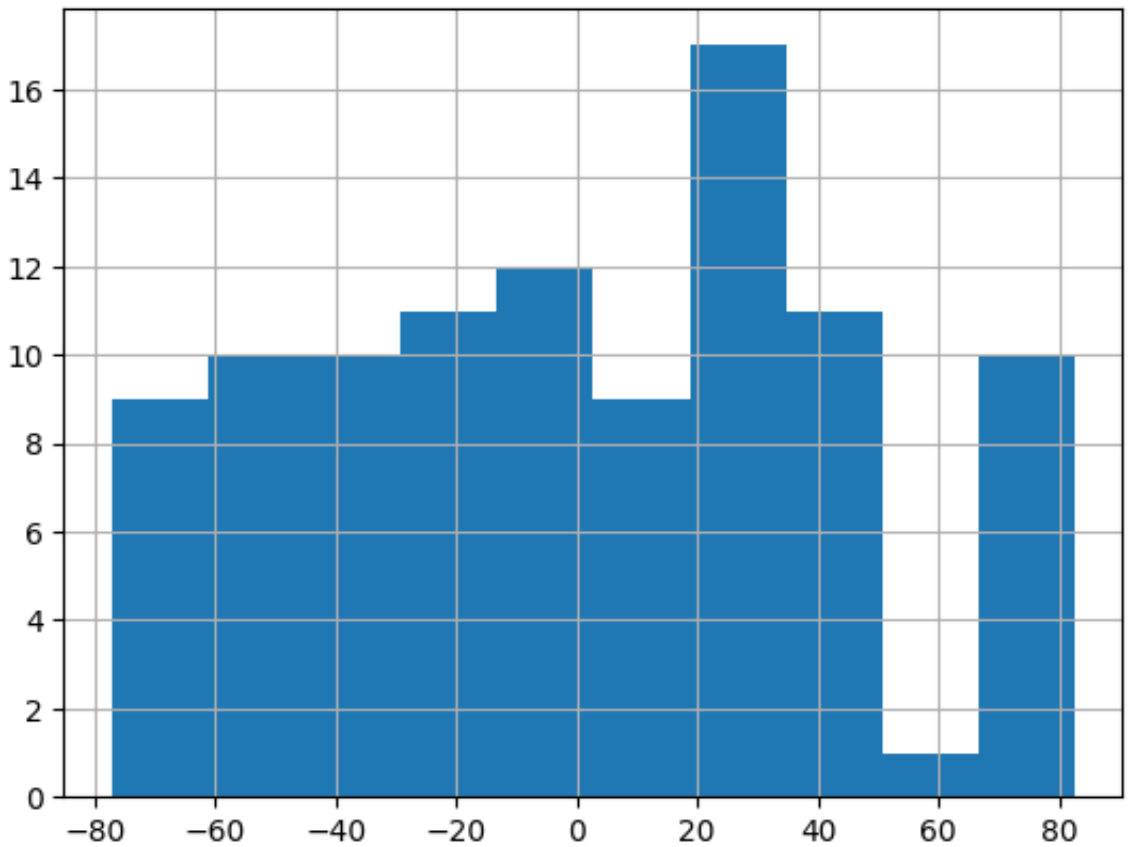
In [10]:
```
print(np.array(pca3.components_[0]).dot(np.array(pca3.components_[1]))
```

```
-1.6653345369377348e-16
```

We can use the projection onto the first principal component as if it were a new feature. For instance, we could plot a histogram of the data against the magnitude of the projection onto the first principal component. In fact, for some purposes, this magnitude (which we have called v) might make a better feature for data analysis than either height, width, or root. We must subtract off the mean before the dot product; the understanding is that this represents the component of the data relative to the mean.

In [11]:
```
((df2 - df2.mean()).dot(np.array(pca.components_[0]))).hist()
```

Out[11]:
```
<Axes: >
```



In [12]:
```
# Showing that this is the same as the value we've called "v"
(df_example["v"] - (df2 - df2.mean()).dot(np.array(pca.components_[0])
```

Out[12]:
```
np.float64(0.0)
```

Linear Regression

Now let's run a linear regression model to predict the root from the height and width, as well as predicting the third principal component from the first and second. Which is easier to predict?

Predicting the root from the height and width

In [13]:
```
linreg = linear_model.LinearRegression()
linreg.fit(df2[['height', 'width']], df2['root'])
preds = linreg.predict(df2[['height', 'width']])
metrics.mean_squared_error(preds, df2['root']) # Mean squared error as
```

Out[13]:
```
138.22487356329611
```

In [14]:
```
np.corrcoef(preds, df2['root']) # Correlation coefficient as a metric
```

Out[14]:
```
array([[1.        , 0.79468295],
       [0.79468295, 1.        ]])
```

Predicting the third principal component from the first and second

In [15]:
```
first_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3
second_principal_component_score = (df2 - df2.mean()).dot(np.array(pca
third_principal_component_score = (df2 - df2.mean()).dot(np.array(pca3
```

In [16]:
```
linreg_first = linear_model.LinearRegression()
pc_X = np.array([first_principal_component_score, second_principal_com
linreg_first.fit(pc_X, third_principal_component_score)
preds = linreg_first.predict(pc_X)
metrics.mean_squared_error(preds, third_principal_component_score) # T
```

Out[16]:
```
78.74928905273173
```

In [17]:
```
np.corrcoef(preds, third_principal_component_score) # More importantly
```

Out[17]:
```
array([[1.00000000e+00, 1.55495941e-14],
       [1.55495941e-14, 1.00000000e+00]])
```

Here, we see that the projections of the first two principal components have no ability to predict the projection of the third, when measured by correlation. This is a special property of principal components. It is intended to ensure that the each component gives new information not present in the previous components.

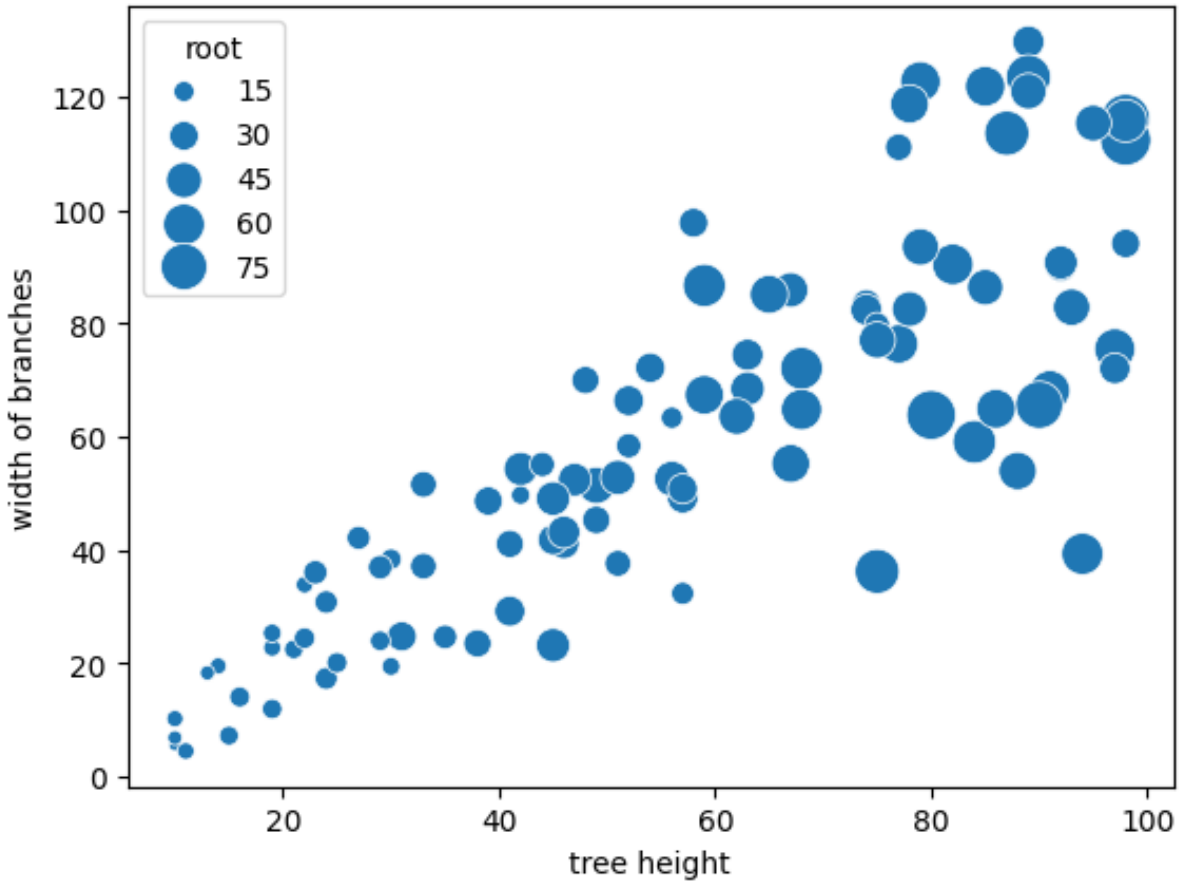
The mean_squared_error is misleading - it makes it look like the error is less for the second computation than the first; but the correlation coefficient is giving more useful information here. We cannot compare the mean squared error for two totally different computations, because mean squared error is highly dependent on the magnitudes of the values in question!

2. Weekly graph question

The tree data above can be graphed via either a pairplot or a bubble plot. What are the advantages and disadvantages of each?

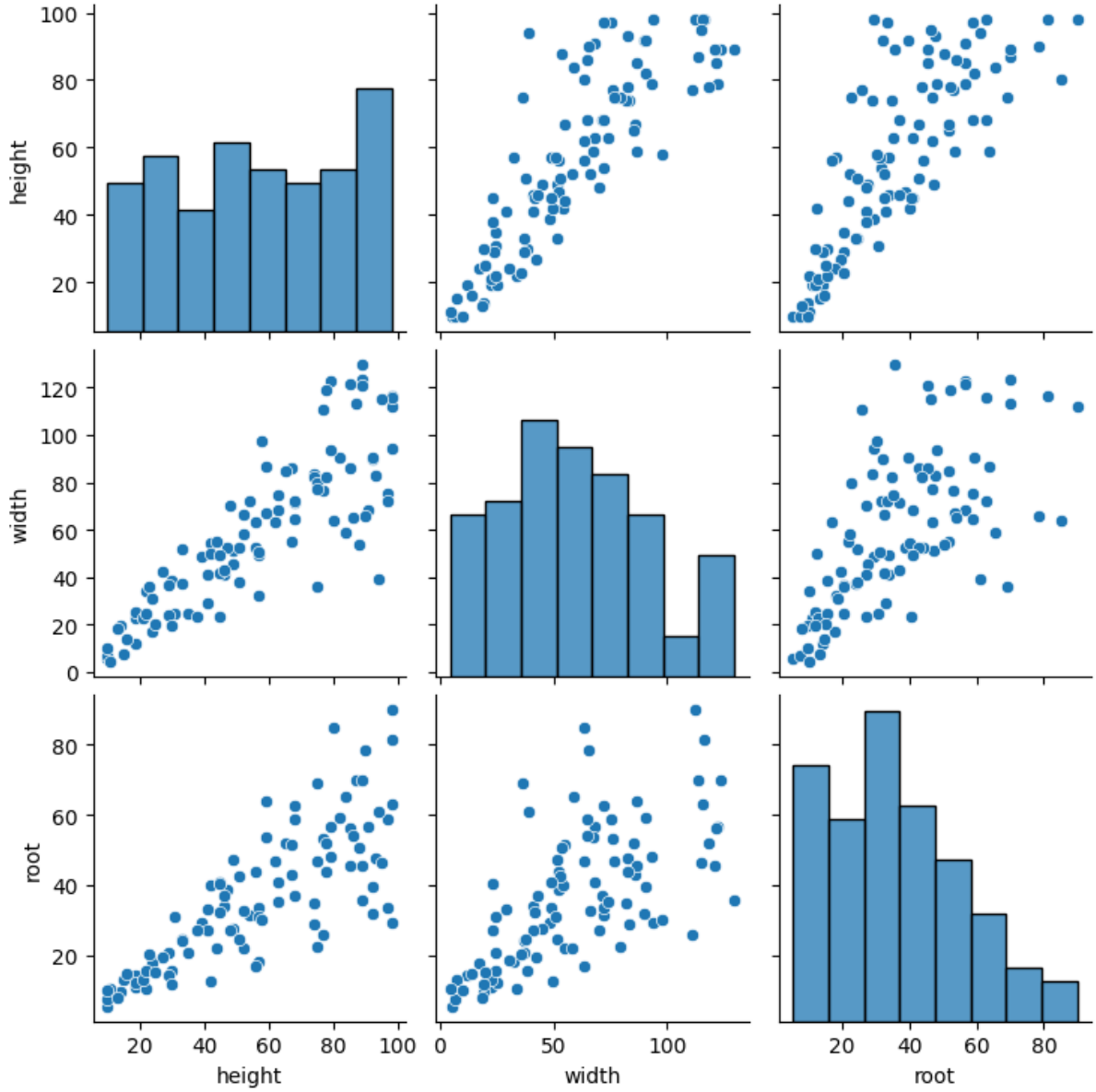
In [18]:
```
ax = sns.scatterplot(data = df2, x = "height", y = "width", size = "ro
ax.set_xlabel("tree height")
ax.set_ylabel("width of branches")
```

Out[18]:
```
Text(0, 0.5, 'width of branches')
```



In [19]:
```
sns.pairplot(df2)
```

Out[19]:
```
<seaborn.axisgrid.PairGrid at 0x23df606d1e0>
```



Pair Plot:

Pros:

- The pair plot includes pairwise relationships between each of the features. It allows us to see histograms of a single feature, and also how the combinations with other features look.

Cons:

- The pair plot does not extend to comparing three features at once. With the bubble plot you can visualize all three together, but the pair plot you need to look at two combinations A,B and B,C and try to extrapolate out to how A,B,C would combine. The other downside is that the pair plot becomes crowded as the number of features grows

Bubble Plot:

Pros:

- The bubble plot visualizes a single relationship between three features at once. In this case, it is helpful to understand at a glance that the feature "root" is correlated with both "width" and "height".

Cons:

- The bubble plot does not include the histograms of each feature, so you would most likely need to plot them separately.

3. Homework - Grocery Store Correlation

In a particular state, the price of milk is highly correlated with the price of cheese. Neither is correlated with the price of bread.

In [20]:
```
num_points = 1000
np.random.seed(0)
milk_series = np.maximum(np.random.normal(scale = 0.5, loc = 2.00, siz
cheese_series = milk_series * np.maximum(np.random.normal(scale = 0.1,
bread_series = np.maximum(np.random.normal(scale = 1.0, loc = 3.00, si
df3 = pd.DataFrame({"milk": milk_series, "cheese": cheese_series, "bre
```

In [21]:
```
df3
```

Out[21]:

	milk	cheese	bread
0	2.882026	3.042256	1.467079
1	2.200079	2.396430	1.288030
2	2.489369	2.384239	3.046135
3	3.120447	3.153122	2.041626
4	2.933779	3.000685	2.919188
...	...	...	...
995	2.206435	2.228003	5.079177
996	1.900801	2.167202	2.092534
997	2.047096	2.079529	2.807596
998	1.426195	1.263337	1.787484
999	1.820943	1.582223	2.919401

1000 rows × 3 columns

Heatmap

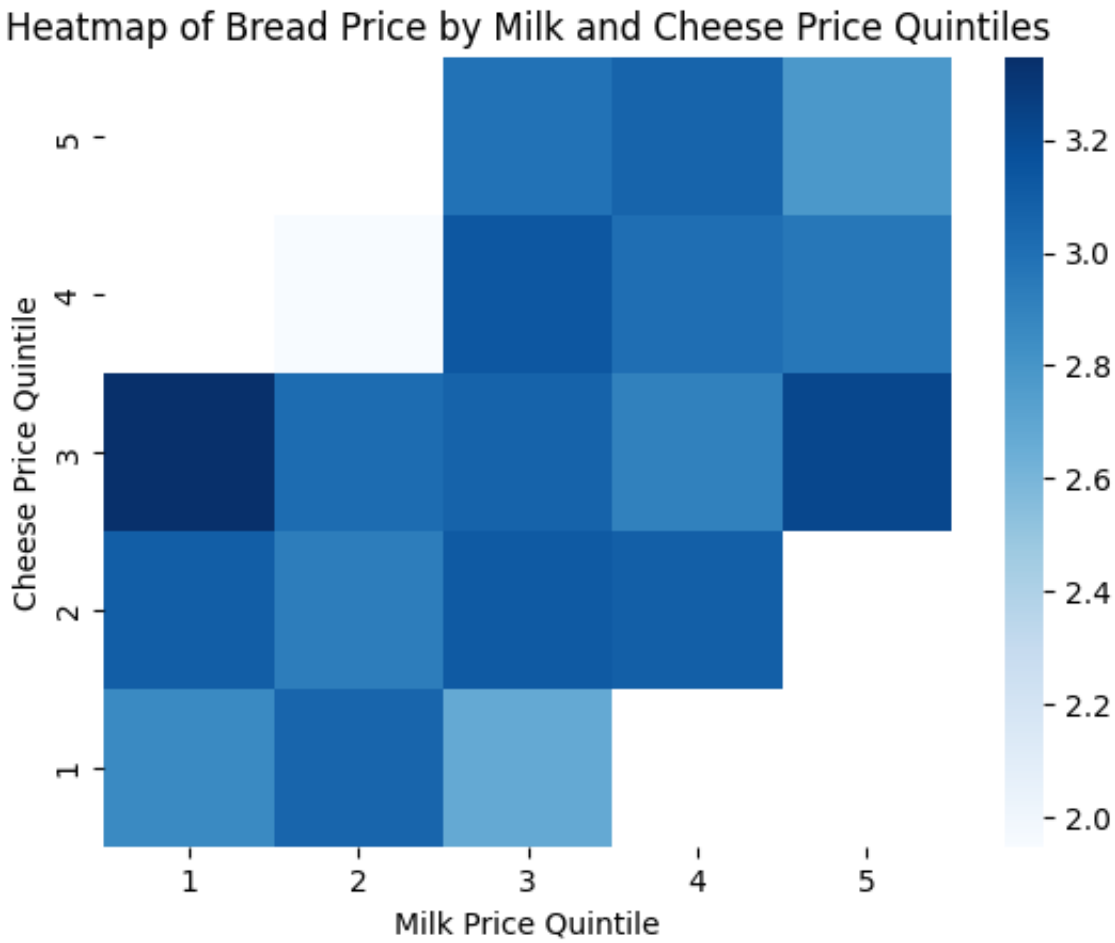
Draw a heatmap where the x-axis category is a discretized version of the milk price and the y-axis category is a discretized version of the cheese price. That is, you can choose "low", "medium", and "high" for the milk price, or, if you prefer, you can choose 1, 2, 3, 4, and 5 for the five quintiles. (The first quintile should include the bottom fifth of the data, the second quintile contains the next fifth, and so on).

Try Googling "how do I change numerical to categorical data" to get advice on this.

In [22]:
```
import matplotlib.pyplot as plt
```

In [23]:
```
df3['milk_bin'] = pd.qcut(df3['milk'], 5, labels=[1, 2, 3, 4, 5])
df3['cheese_bin'] = pd.qcut(df3['cheese'], 5, labels=[1, 2, 3, 4, 5])

df3_pivot = df3.pivot_table(index='cheese_bin', columns='milk_bin', va

sns.heatmap(df3_pivot, cmap='Blues')
plt.title('Heatmap of Bread Price by Milk and Cheese Price Quintiles')
plt.xlabel('Milk Price Quintile')
plt.ylabel('Cheese Price Quintile')
plt.gca().invert_yaxis()
None
```

```
C:\Users\caleb\AppData\Local\Temp\ipykernel_7388\3251682398.py:4: Futur
eWarning: The default value of observed=False is deprecated and will ch
ange to observed=True in a future version of pandas. Specify observed=F
alse to silence this warning and retain the current behavior
  df3_pivot = df3.pivot_table(index='cheese_bin', columns='milk_bin', v
alues='bread', aggfunc='mean')
```

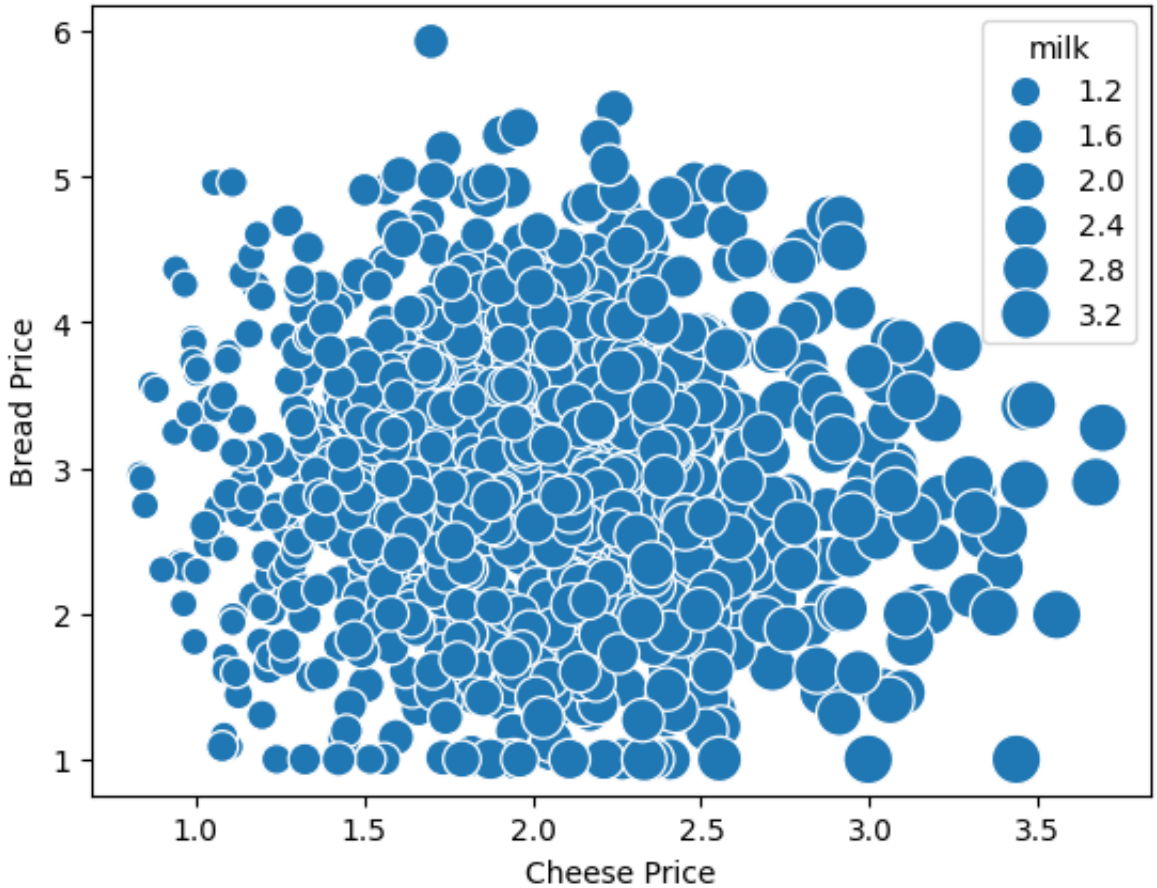


Bubble plot

Try all three possible bubble plots: one where the size of the dot is determined by the milk price, one where it's the cheese price, and one where it's the bread price. Which of these plots is / are the most informative and useful?

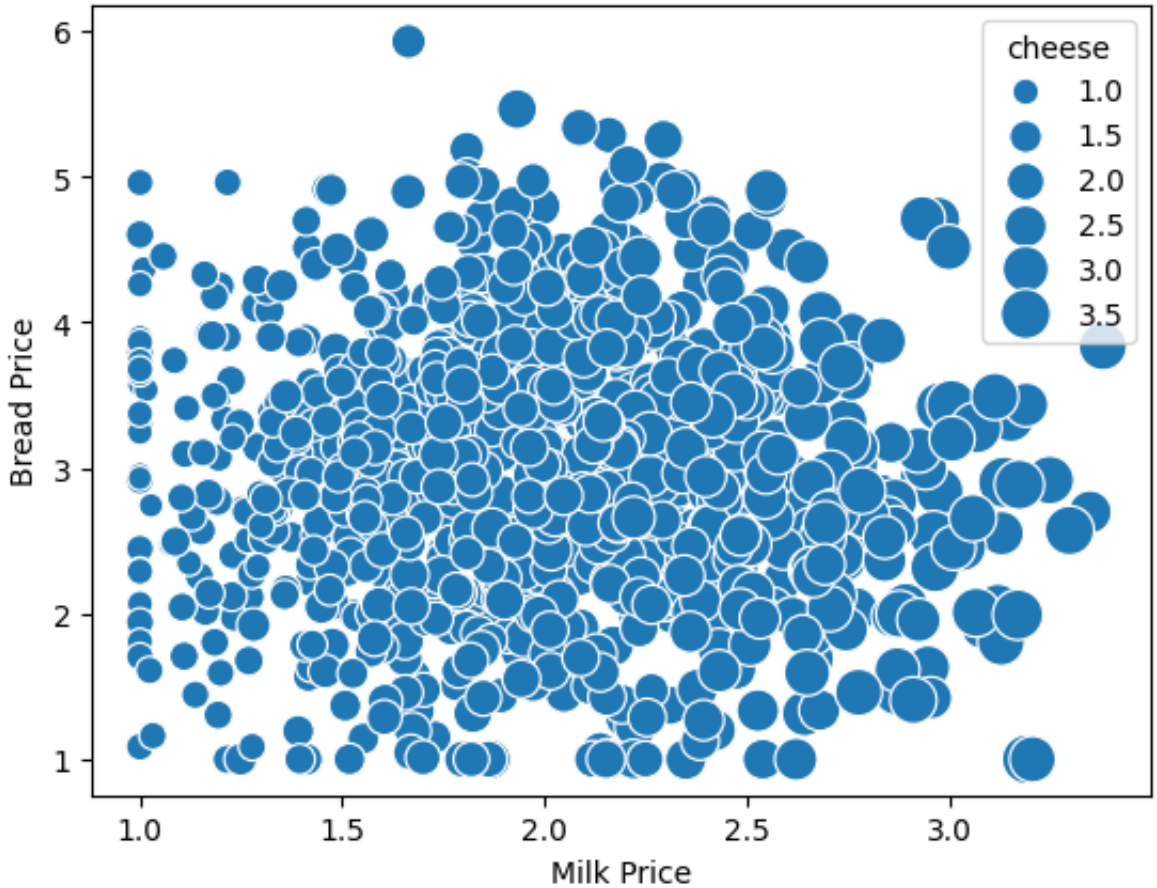
In [24]:
```
# Dot is milk price
minroot = milk_series.min()
maxroot = milk_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df3, x = "cheese", y = "bread", size = "mi
ax.set_xlabel("Cheese Price")
ax.set_ylabel("Bread Price")
```

Out[24]:
```
Text(0, 0.5, 'Bread Price')
```



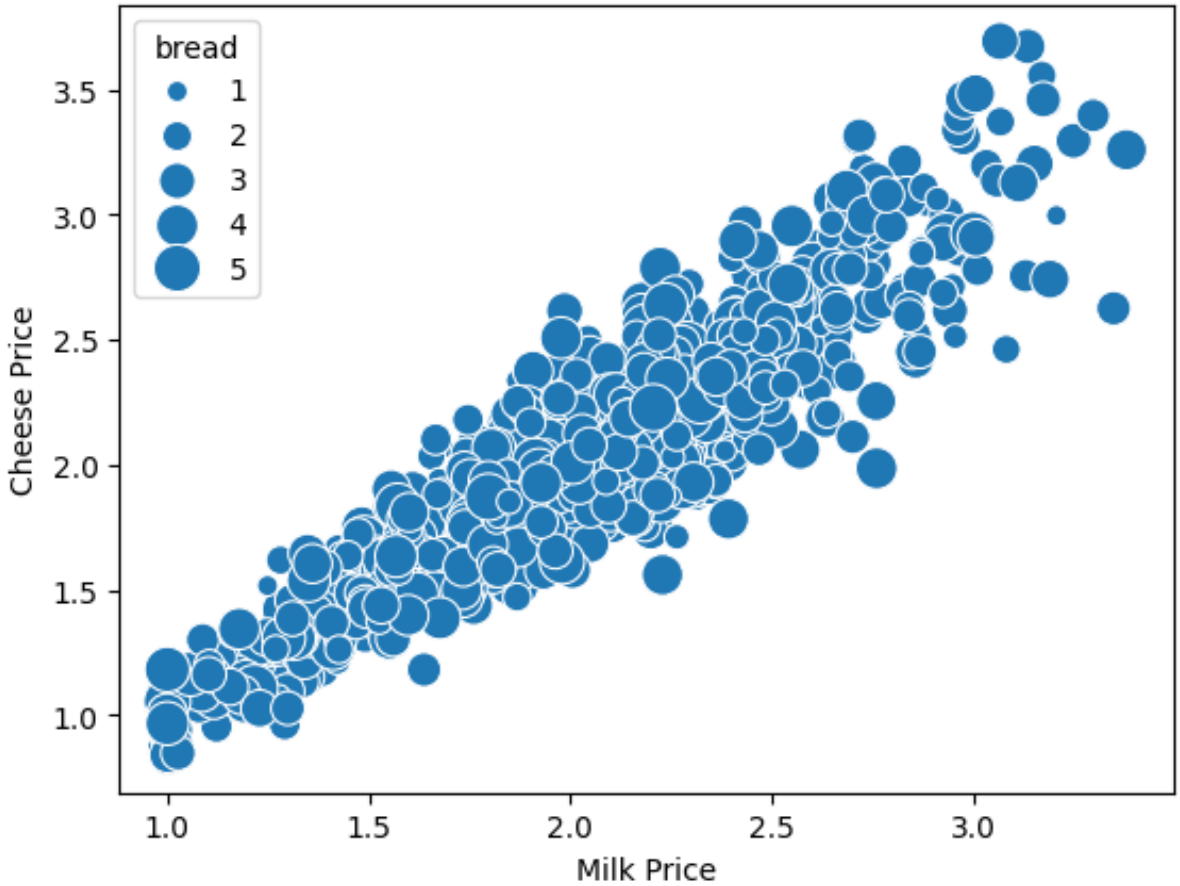
In [25]:
```
# Dot is Cheese price
minroot = cheese_series.min()
maxroot = cheese_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df3, x = "milk", y = "bread", size = "chee
ax.set_xlabel("Milk Price")
ax.set_ylabel("Bread Price")
```

Out[25]: Text(0, 0.5, 'Bread Price')



In [26]:
```
# Dot is Bread price
minroot = bread_series.min()
maxroot = bread_series.max()
maxbubble = 300
ax = sns.scatterplot(data = df3, x = "milk", y = "cheese", size = "bre
ax.set_xlabel("Milk Price")
ax.set_ylabel("Cheese Price")
```

Out[26]:
```
Text(0, 0.5, 'Cheese Price')
```



The most informative plot is the bubble plot with the Bread Price as the dot size. The other two versions of the plot are a cloud of noise with no patterns. We can see a strong correlation between the Milk and Cheese prices, and do not see a strong correlation between those and the Bread Price due to the dot size looking random.

Principal component analysis

Perform a principal component analysis of the data. What are the first, second, and third component vectors?

Challenge question: explain why the three vectors point as they do. (Could someone have guessed the order of the three vectors, even before running the analysis?)

In [27]:
```
pca = decomposition.PCA(n_components = 3)
pca.fit(df3)

print(f"The first principal component explains {pca.explained_variance
print(f"The second principal component explains {pca.explained_varianc
print(f"The third principal component explains {pca.explained_variance
print(f"The components_ vector is {pca.components_}")
```

```
The first principal component explains 0.7829444272057101 of the varian
ce
The second principal component explains 0.16324412274814487 of the vari
ance
The third principal component explains 0.04522838893323079 of the varia
nce
The components_ vector is [[ 0.22299336  0.23834686 -0.01069345  0.6683
7654  0.66829873]
 [-0.00740863 -0.00431591  0.99983153  0.00438285  0.01562627]
 [ 0.15322251 -0.14385202  0.00828586  0.69140406 -0.6911738 ]]
```

Since the Milk and Cheese prices are so highly correlated, the first component can explain the majority of the variance of both. The second component likely explains the Bread price variance and the final component the remaining variance from the Milk or Cheese prices.

Linear regression

Use linear regression to predict the milk price from the other two, the cheese price from the other two, and the bread price from the other two. Which prediction(s) work the best, and why?

In [28]:
```
# Predict Milk Price
linreg = linear_model.LinearRegression()
linreg.fit(df3[['cheese', 'bread']], df3['milk'])
preds = linreg.predict(df3[['cheese', 'bread']])
metrics.mean_squared_error(preds, df3['milk']) # Mean squared error as
```

Out[28]: 0.03447410921882385

In [29]:
```
# Predict Cheese Price
linreg = linear_model.LinearRegression()
linreg.fit(df3[['milk', 'bread']], df3['cheese'])
preds = linreg.predict(df3[['milk', 'bread']])
metrics.mean_squared_error(preds, df3['cheese']) # Mean squared error
```

Out[29]: 0.0396561665666383

In [30]:
```
# Predict Bread Price
linreg = linear_model.LinearRegression()
linreg.fit(df3[['milk', 'cheese']], df3['bread'])
preds = linreg.predict(df3[['milk', 'cheese']])
metrics.mean_squared_error(preds, df3['bread']) # Mean squared error a
```

Out[30]: 0.8772889513306434

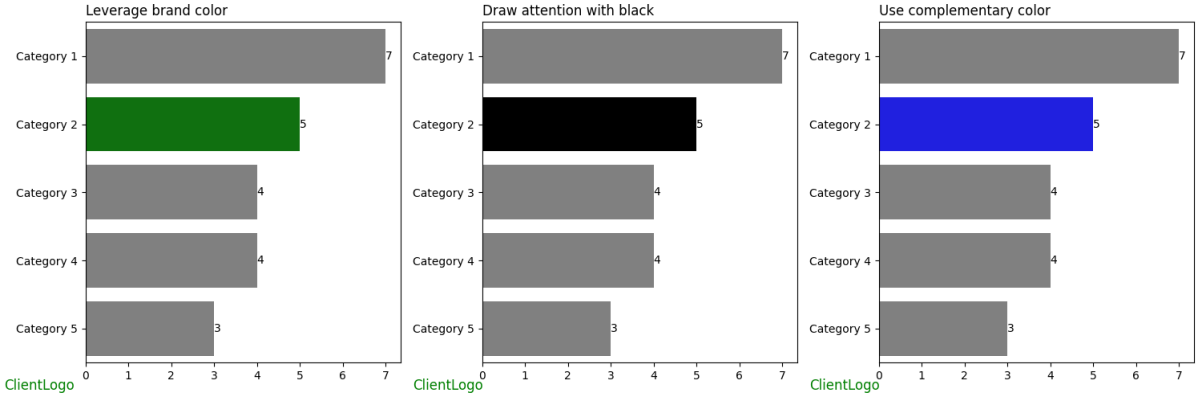
Predicting Milk from the Cheese and Bread price works the best, closely followed by predicting Cheese from the Milk and Bread price. We know that the Milk and

Cheese price vary together, so they can be used to predict each other.

4. Storytelling With Data plot

Reproduce any graph of your choice in p. 120-135 of the Storytelling With Data book as best you can. ("Use color consistently" in chapter four up to "Create a clear visual hierarchy of information" in chapter five.) You do not have to get the exact data values right, just the overall look and feel.

In [87]:
```
# Figure 4.16 (Page 124)
df_4 = pd.DataFrame(
    [[7,5,4,4,3]],
    columns=['Category 1', 'Category 2', 'Category 3', 'Category 4', '
)

fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(15, 5), sharey=True

first_colors = ['gray' if col != 'Category 2' else 'green' for col in
second_colors = ['gray' if col != 'Category 2' else 'black' for col in
third_colors = ['gray' if col != 'Category 2' else 'blue' for col in d

sns.barplot(data=df_4, orient = 'h', palette=first_colors, ax=ax1)
ax1.set_title("Leverage brand color", loc='left')
plt.figtext(0.0, 0.01, "ClientLogo", fontsize=12, color="green")

sns.barplot(data=df_4, orient = 'h', palette=second_colors, ax=ax2)
ax2.set_title("Draw attention with black", loc='left')
plt.figtext(0.34, 0.01, "ClientLogo", fontsize=12, color="green")

sns.barplot(data=df_4, orient = 'h', palette=third_colors, ax=ax3)
ax3.set_title("Use complementary color", loc='left')
plt.figtext(0.67, 0.01, "ClientLogo", fontsize=12, color="green")

for container in ax1.containers:
    ax1.bar_label(container)
for container in ax2.containers:
    ax2.bar_label(container)
for container in ax3.containers:
    ax3.bar_label(container)
ax1.tick_params(labelleft=True)
ax2.tick_params(labelleft=True)
ax3.tick_params(labelleft=True)
plt.tight_layout()
```


Leverage brand color
Category 1
Category 2
Category 3
Category 4
Category 5
7
5
4
4
3
0 1 2 3 4 5 6 7
ClientLogo
Draw attention with black
Category 1
Category 2
Category 3
Category 4
Category 5
7
5
4
4
3
0 1 2 3 4 5 6 7
ClientLogo
Use complementary color
Category 1
Category 2
Category 3
Category 4
Category 5
7
5
4
4
3
0 1 2 3 4 5 6 7
ClientLogo

Week 9 - Multivariate Analysis, part 2

1. Lesson - no lesson this week

2. Weekly graph question

The graph below plots the first two principal component scores in a scatter plot. What can be said about the three outliers in the upper left corner of the graph? Is their first principal component score high or low? What about their second principal component score? What does that mean about their values in series_1, series_2, and series_3? It seems to me that you can say something about series_3 (what can you say?) but you may have a harder time saying something about series_1 and series_2, and an almost impossible time saying anything about the relative values of series_1 and series_2. Why is that? How are series_1 and series_2 related, according to how they were created? If you like, try drawing a pairplot for all three series and see what you get.

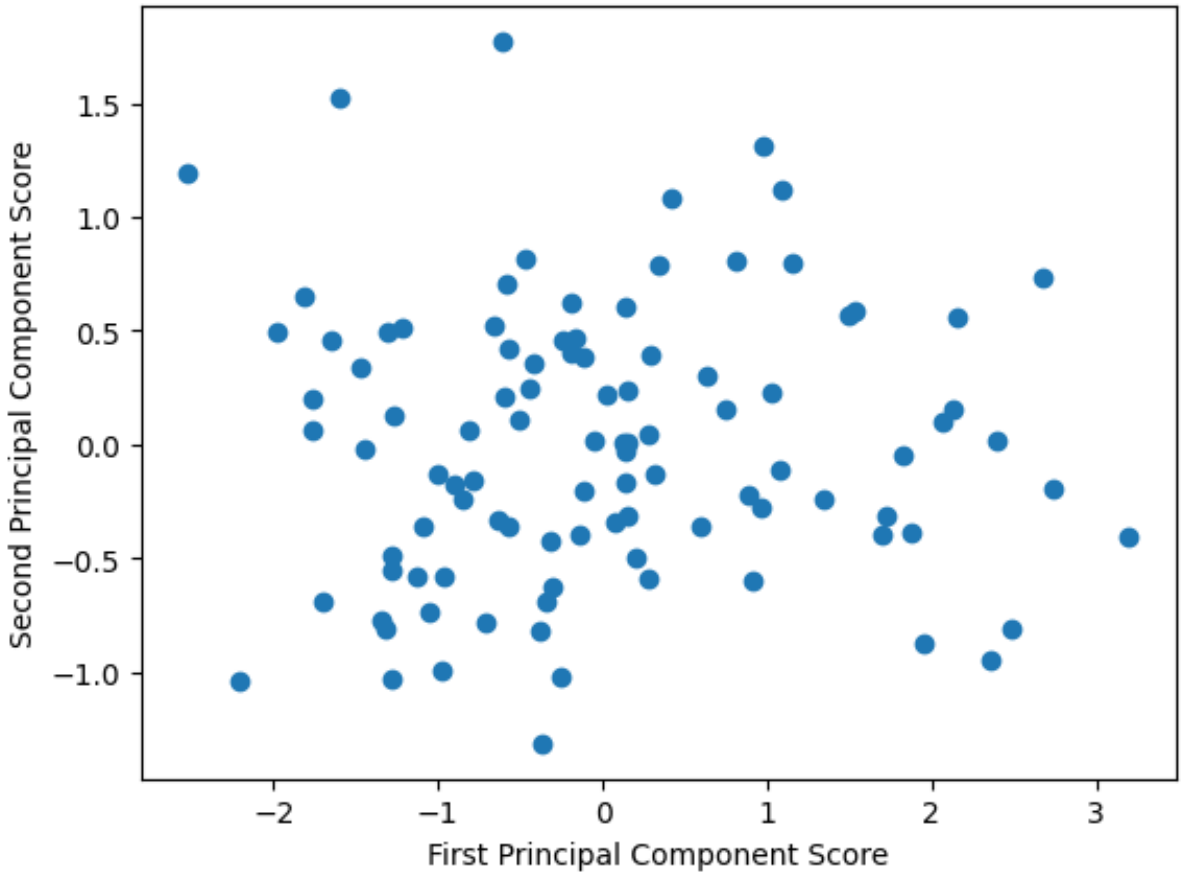
Overall, what are the advantages and disadvantages of the graph below? Does it show anything interesting?

In [1]:
```
import numpy as np
import pandas as pd
from sklearn import decomposition
import matplotlib.pyplot as plt

np.random.seed(0)
num_points = 100
series_1 = np.random.normal(loc = 2, scale = 0.5, size = num_points)
series_2 = series_1 * (1 + np.random.normal(loc = 0, scale = 0.1, size
series_3 = series_1 * (1 + np.random.normal(loc = 0, scale = 0.5, size
df = pd.DataFrame({'ser1': series_1, 'ser2': series_2, 'ser3': series_
df = df - df.mean() # set mean to zero, so we don't have to subtract m

pca3 = decomposition.PCA(n_components = 3)
pca3.fit(df)
print(pca3.explained_variance_ratio_)
print(pca3.components_)

first_principal_component_score = df.dot(pca3.components_[0])
second_principal_component_score = df.dot(pca3.components_[1])
plt.scatter(first_principal_component_score, second_principal_componen
```

```
plt.xlabel("First Principal Component Score")
plt.ylabel("Second Principal Component Score")
```

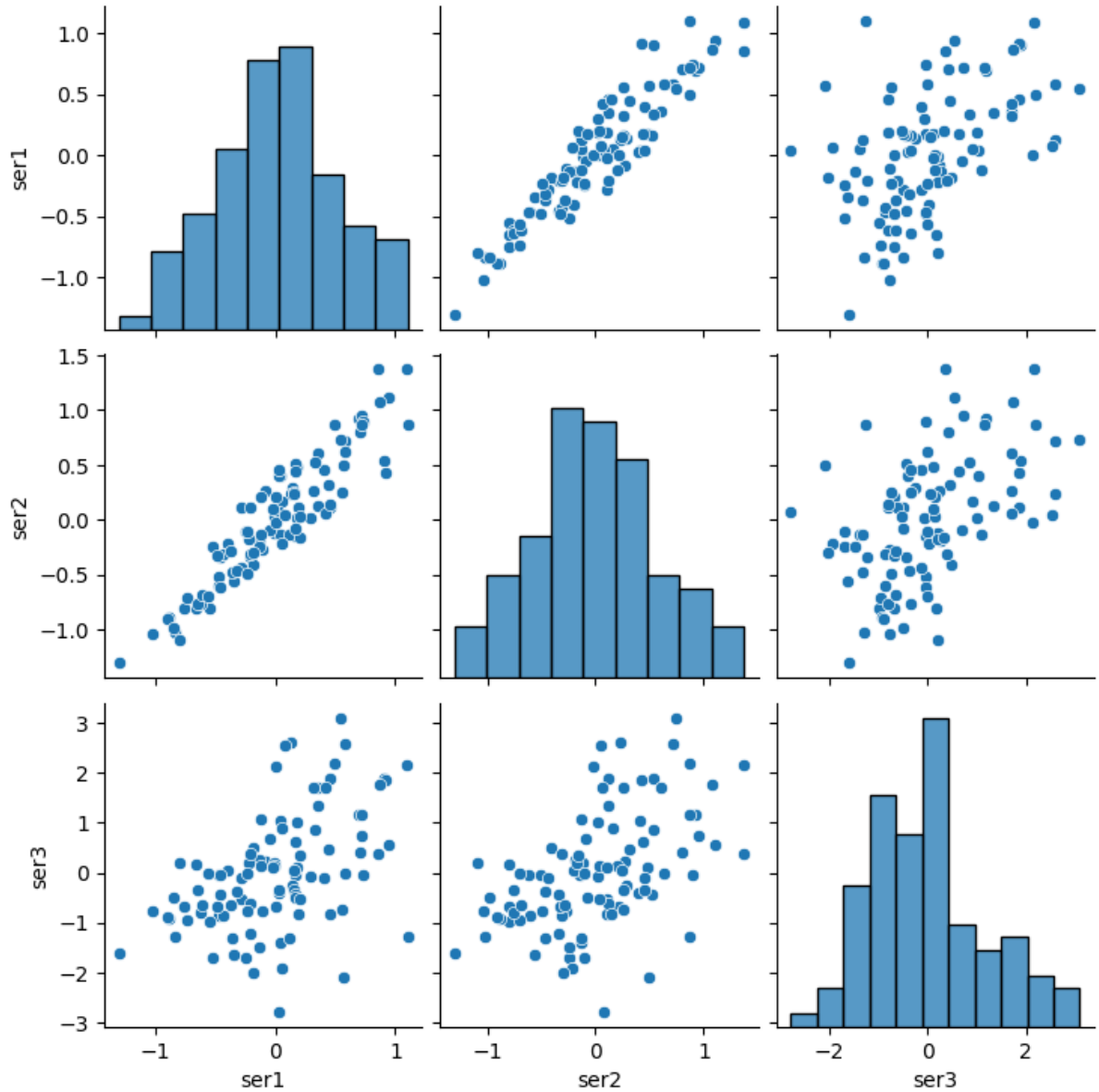
```
[0.79916477 0.18990532 0.01092991]
[[ 0.26541493  0.30096233  0.91595665]
 [ 0.60337553  0.6891417  -0.40127506]
 [ 0.75199261 -0.65917023 -0.00131519]]
```

Out[1]: Text(0, 0.5, 'Second Principal Component Score')



In [2]:
```
import seaborn as sns
sns.pairplot(df)
```

Out[2]: <seaborn.axisgrid.PairGrid at 0x2321810c040>



The three outliers in the upper left corner of the graph each have a low first principal component score (between -3 and 0) and a high second principal component score (between 1 and 2). We also know from the pair plots that series1 and series2 are highly correlated. Therefore, since the first principal component captures the direction of maximum variance, it will capture the relationship between these two series. These outliers likely have low values in series1 and series2. Next, the second principal component will capture the remaining variance in series3. Since the second principal component has a high value for these outliers, the values in the series3 is likely also to be high. The second principal component shows individidual variation of series3, whereas the first principal component shows a joint variance of series1 and series2.

One challenge with the graph is that it is not easily digestible. I had to utilize the pairplot to first understand the correlations between the series, before making judgement about the PCA results. Additionally, it would help to highlight the

interesting data points with a different color to make them stick out. The advantage of this graph is that it successfully identifies three outliers.

3. Working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do: Draw heatmaps.

Draw bubble plots.

Perform Principal Component Analysis to find out the directions in which the data varies. Can you represent the data using only its projection onto its first principal component, using the methods described in Week 8? How much of the variance would this capture?

Try performing linear regression analysis using different sets of features. Which features seem most likely to be useful to predict other features?

Conclusions: Explain what conclusions you would draw from this analysis: are the data what you expect? Are the data likely to be usable? If the data are not useable, find some new data!

Do you see any outliers? (Data points that are far from the rest of the data).

Does the Principal Component Analysis suggest a way to represent the data using fewer dimensions than usual - using its first one or two principal component scores, perhaps?

Try using your correlation information from previous weeks to help choose features for linear regression.

In [3]:
```
import pandas as pd
import seaborn as sns
```

In [4]:
```
df = pd.read_csv('C:/Users/caleb/Downloads/ppploan/ppp_over_150k.csv')
```

In [6]:
```
df
```

Out[6]:

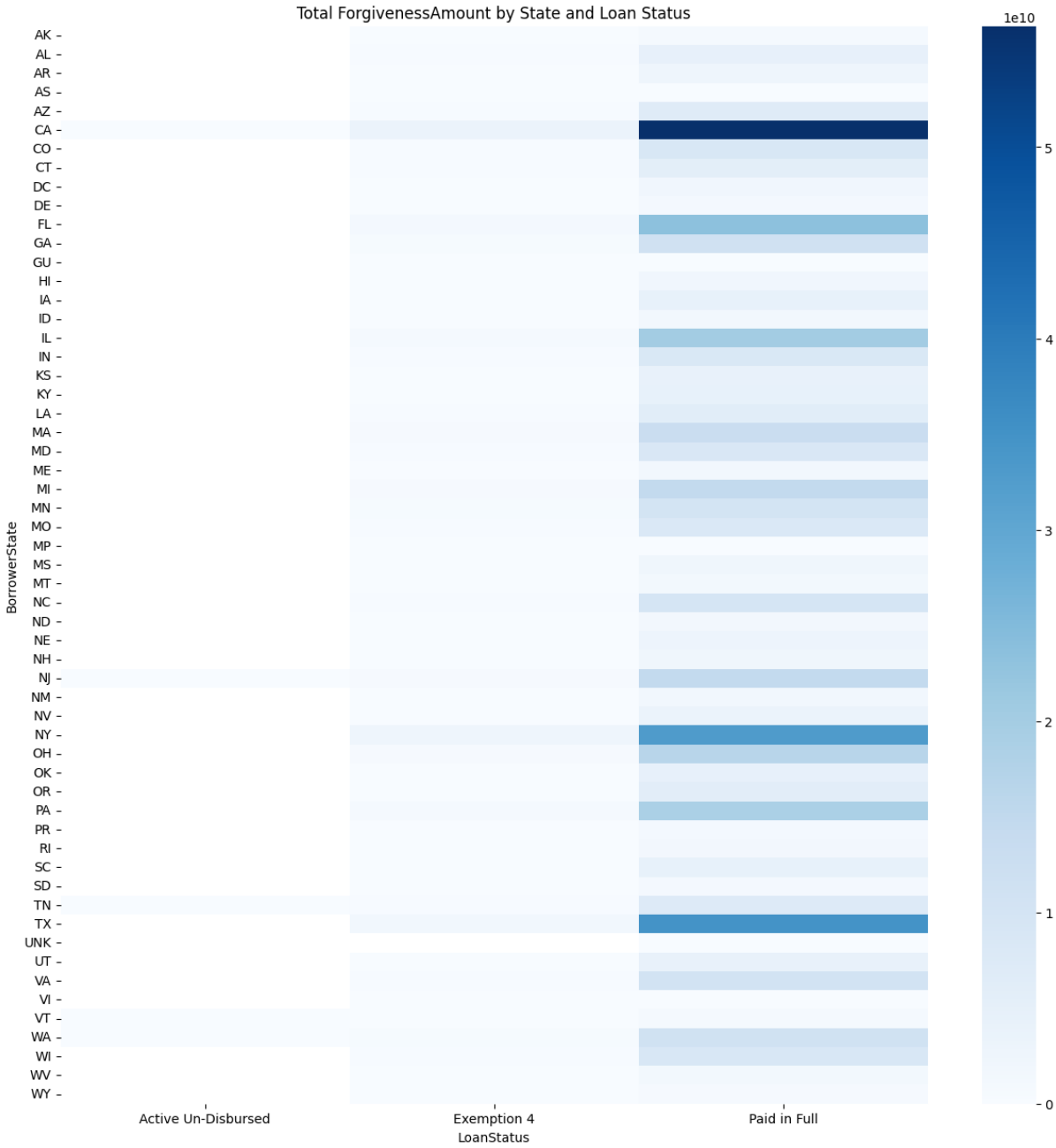
	LoanNumber	DateApproved	SBAOfficeCode	ProcessingMethod	Borro
0	9547507704	2020-05-01	464	PPP	c
1	9777677704	2020-05-01	464	PPP	pleas
2	5791407702	2020-05-01	1013	PPP	boye
3	6223567700	2020-05-01	920	PPP	co
4	9662437702	2020-05-01	101	PPP	a
...	...	...	...	...	
968527	4395967002	2020-04-03	897	PPP	roy e
968528	6985647108	2020-04-14	897	PPP	s co deve
968529	7996438405	2021-02-12	897	PPS	s w
968530	9054647103	2020-04-15	897	PPP	ede
968531	9184687004	2020-04-09	897	PPP	s ente

968532 rows × 66 columns

In [15]:
```
pivot = df.pivot_table(index="BorrowerState", columns="LoanStatus", va
```

In [20]:
```
plt.figure(figsize=(15, 15)) # Set the desired width and height in inc
ax = sns.heatmap(pivot, cmap = "Blues")
ax.set_title("Total ForgivenessAmount by State and Loan Status")
```

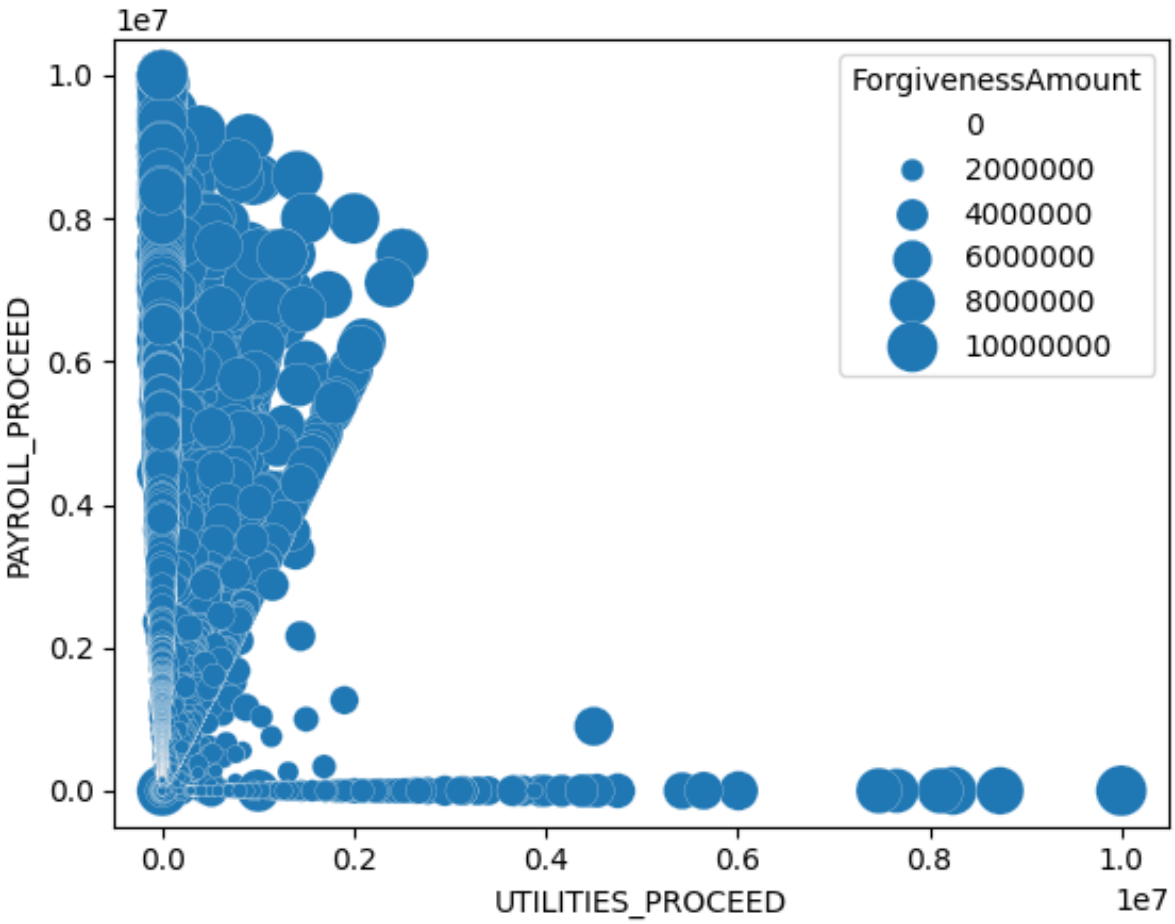
Out[20]:
```
Text(0.5, 1.0, 'Total ForgivenessAmount by State and Loan Status')
```



In [27]:
```
df_cleaned = pd.read_csv('C:/Users/caleb/Downloads/ppploan/ppp_cleaned
```

In [32]:
```
# 'UTILITIES_PROCEED', 'PAYROLL_PROCEED', 'MORTGAGE_INTEREST_PROCEED',
minroot = df_cleaned['MORTGAGE_INTEREST_PROCEED'].min()
maxroot = df_cleaned['MORTGAGE_INTEREST_PROCEED'].max()
maxbubble = 300
ax = sns.scatterplot(data = df_cleaned, x = "UTILITIES_PROCEED", y = "
ax.set_xlabel("UTILITIES_PROCEED")
ax.set_ylabel("PAYROLL_PROCEED")
```

Out[32]:
```
Text(0, 0.5, 'PAYROLL_PROCEED')
```



In [ ]:
```
# 1. Determine the minimum class size
min_size = df_cleaned.groupby('Fraud')['Fraud'].count().min()
# print(f"Minimum class size: {min_size}") # Should be around 100 for

# 2. Group by the target column and sample 'n' items from each group
# The 'sample' method on a grouped object samples from each group auto
balanced_df_under = df_cleaned.groupby('Fraud').sample(n=min_size, ran

# Optional: shuffle the resulting DataFrame as groupby sample preserve
balanced_df_under = balanced_df_under.sample(frac=1, random_state=42).
```

In [36]:
```
balanced_df_under
```

Out[36]:

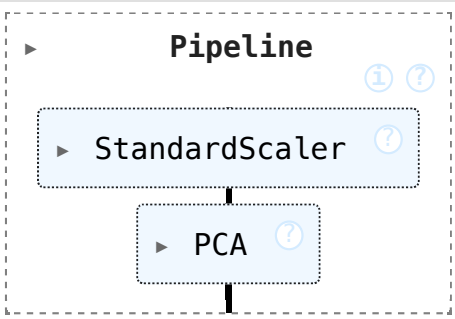
	Unnamed: 0	ProcessingMethod	BorrowerState	LoanStatus	Term	InitialApp
0	201060	0.0	8.0	1.0	24	
1	798813	0.0	46.0	1.0	24	
2	918753	0.0	50.0	1.0	60	
3	295246	0.0	13.0	2.0	24	
4	814054	0.0	47.0	1.0	5	
...	...	...	...	...	...	
185	827885	0.0	47.0	1.0	5	
186	382304	1.0	19.0	1.0	60	
187	49695	0.0	5.0	2.0	24	
188	782910	0.0	44.0	1.0	46	
189	903307	0.0	50.0	2.0	24	

190 rows × 43 columns

In [43]:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ('scaling', StandardScaler()),
    ('pca', PCA(n_components=3))
])
pipeline.fit(balanced_df_under.drop(columns=['Fraud']), balanced_df_un
pipeline
```

Out[43]:



In [46]:

```
print(pipeline[1].explained_variance_ratio_)
```

```
[0.13868966 0.13317692 0.10122726]
```

In [47]:

```
print(pipeline[1].components_)
```

```
[[ 6.63463002e-02 -1.15641493e-01  7.58676499e-02 -1.87374511e-01
  -8.13649800e-02  3.59683085e-01  3.61550110e-01  2.02096206e-02
   8.77753817e-02 -4.02256630e-02 -3.18768434e-03  4.37774238e-02
   7.58676499e-02  2.13564933e-01  9.41070301e-02  3.57221936e-01
  -5.09465826e-02  8.15388626e-02 -0.00000000e+00 -9.50873473e-03
   2.16297767e-02  2.89646597e-02  1.13284159e-02  7.52907416e-02
  -5.17776527e-02  2.51230115e-02 -1.63252115e-01  3.48445904e-01
  -2.01473227e-01  3.61550264e-01 -9.67343392e-02  4.39636439e-02
  -2.86019562e-02 -5.14791433e-02  2.49906284e-02 -0.00000000e+00
  -3.27475753e-02  1.98624581e-02  1.69947549e-01 -9.63850585e-02
  -2.01809056e-01 -2.00715736e-01]
 [-1.28847027e-01 -1.38353728e-02 -1.23388286e-01  3.00326290e-01
  -3.10522053e-02  1.35014232e-01  1.62836614e-01  8.89692259e-03
   1.89806247e-02  8.59062188e-02  5.98826229e-02 -6.76114126e-02
  -1.23388286e-01  2.45326467e-01  1.34123723e-01  1.36558199e-01
   6.01856996e-02  2.15860608e-01 -0.00000000e+00  2.02090321e-01
   1.87532512e-01 -1.42371066e-02  3.32913784e-03 -5.44101728e-02
  -1.33341093e-02  3.33120969e-01 -5.04912129e-03 -1.47734093e-01
   3.00386936e-01  1.62836643e-01 -2.49111904e-02  1.33044443e-01
  -2.19794904e-01  6.07626806e-02  1.70952860e-01 -0.00000000e+00
   1.46681804e-01  1.82202555e-01 -6.98837082e-02 -6.02239773e-02
   3.00213349e-01  2.68339722e-01]
 [ 4.22815992e-02  3.21703372e-01  5.42848266e-02  7.79360687e-02
   3.20335329e-01  1.55242816e-01  1.56681150e-01 -9.25847453e-02
  -9.27637704e-02 -3.78404302e-02 -2.84763143e-03 -8.44120155e-02
   5.42848266e-02  6.32464687e-02 -1.89594113e-01  1.88176218e-01
  -1.12801272e-01 -1.59680659e-01 -3.30872245e-24 -8.33920492e-02
  -6.02595793e-02 -1.19293702e-02 -9.72088011e-02 -1.34894332e-01
  -6.39735609e-02  1.80734516e-01 -6.91498758e-02 -9.80000576e-03
   1.07547853e-01  1.56680571e-01  2.52126291e-01 -2.48300763e-01
   3.09886267e-01 -1.13950277e-01 -2.27124670e-01 -0.00000000e+00
  -8.66646752e-02 -7.08424473e-02  8.26448183e-02  3.27375409e-01
   1.08514266e-01  2.27841761e-01]]
```

We can explain about 33 percent of the data with 3 principal components based on this PCA analysis

4. Storytelling With Data plot

Reproduce any graph of your choice in p. 136-150 of the Storytelling With Data book as best you can. ("The power of super-categories" to the end of chapter 5). You do not have to get the exact data values right, just the overall look and feel.

In [78]:
```
import matplotlib.pyplot as plt
```

In [55]:
```
seg_1_us = 16
seg_2_us = 7
seg_3_us = 10
seg_4_us = 10
```

```
seg_5_us = 10
seg_6_us = 32
seg_7_us = 15

seg_1_customers = 9
seg_2_customers = 10
seg_3_customers = 15
seg_4_customers = 18
seg_5_customers = 17
seg_6_customers = 20
seg_7_customers = 11

df = pd.DataFrame(
    {
        'Customer Base': ['US Population', 'Our Customers'],
        'Segment 1': [16, 9],
        'Segment 2': [7,10],
        'Segment 3': [10,15],
        'Segment 4': [10,18],
        'Segment 5': [10,17],
        'Segment 6': [32,20],
        'Segment 7': [15,11],
    }
)
df
```

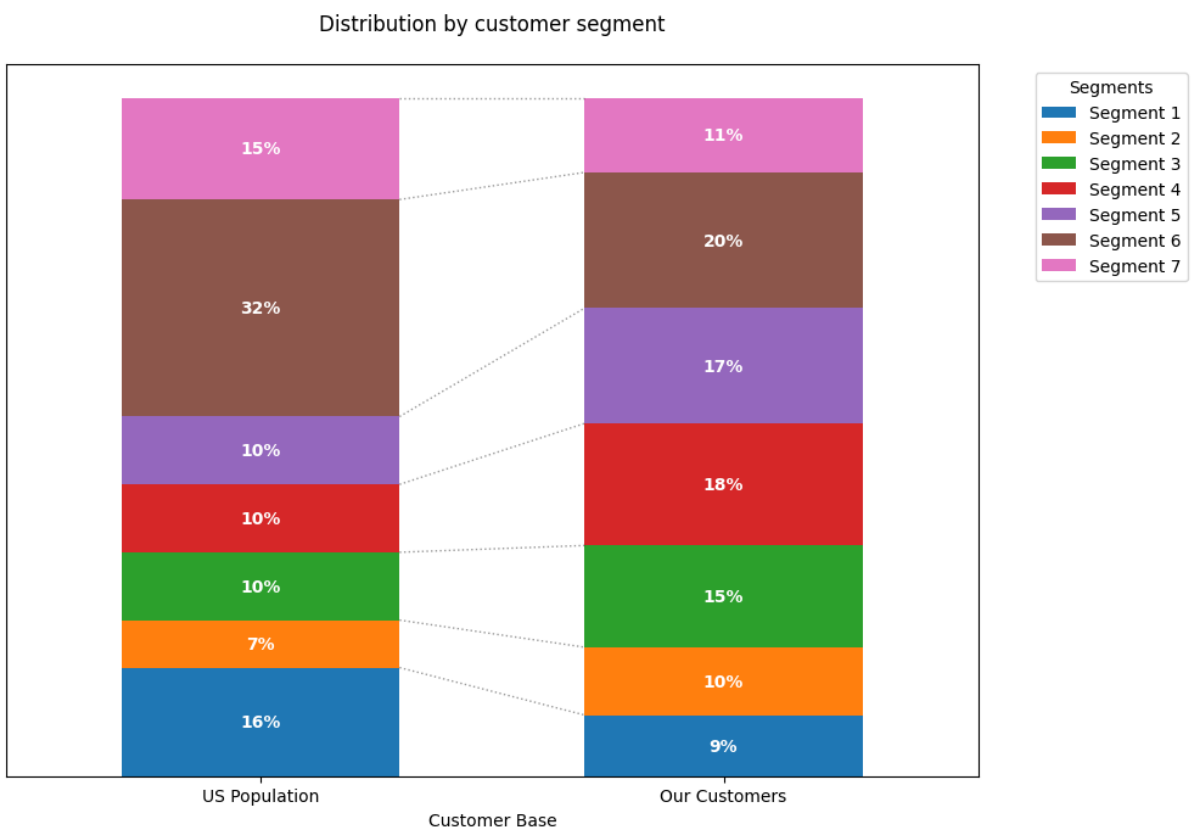
Out[55]:

	Customer Base	Segment 1	Segment 2	Segment 3	Segment 4	Segment 5	Segment 6	Seg
0	US Population	16	7	10	10	10	32	
1	Our Customers	9	10	15	18	17	20	

In [ ]:

```
# Figure 5.12 (p. 147)
ax = df.set_index('Customer Base').plot(kind='bar', stacked=True, figs
patches = ax.patches
num_segments = len(df.columns) - 1

for i in range(num_segments):
    # Index for Bar 1 and Bar 2 of the same segment
    idx1 = i * 2
    idx2 = i * 2 + 1

    rect1 = patches[idx1]
    rect2 = patches[idx2]

    # Top y-coordinate of this segment for both bars
    y1_top = rect1.get_y() + rect1.get_height()
    y2_top = rect2.get_y() + rect2.get_height()
```

```
    ax.plot([rect1.get_x() + rect1.get_width(), rect2.get_x()],
            [y1_top, y2_top],
            color='black', linestyle=':', linewidth=1, alpha=0.4)

    # --- ADD PERCENTAGE LABELS ---
    for rect in [rect1, rect2]:
        h = rect.get_height()
        if h > 0:
            ax.text(rect.get_x() + rect.get_width()/2,
                    rect.get_y() + h/2,
                    f'{h:.0f}%',
                    ha='center', va='center', color='white', fontweigh

plt.title('Distribution by customer segment', pad=20)
plt.xticks(rotation=0)
ax.set_yticks([])
plt.legend(title='Segments', bbox_to_anchor=(1.05, 1), loc='upper left
plt.tight_layout()
```



Week 10 - Introduction to Modeling

1. Lesson

Last week, we reviewed one model: linear regression. This week, we are going to review another, called random forest. The random forest model builds many decision trees, then averages their estimates to make a prediction.

You will also perform:

- Cross-validation
- Hyperparameter tuning
- A performance metric (root mean squared error)

How does random forest work?

A random forest is a set of decision trees. Let's say we use n_estimators = 100 decision trees, and we have ten features. Then each decision tree is built so that a given branch gets about three (sqrt(10)) features to choose from. It starts from the top of the tree, selects the best feature, then proceeds to the next nodes. The best feature is the one that minimizes the mean squared error, which must be computed for both branches, weighted according to the probability of each branch, and then summed over both branches.

In [1]:
```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import root_mean_squared_error
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer
```

Here, we build three series of random numbers between 0 and 1. We use a somewhat arbitrary formula to construct the synthetic data. The fourth value (the target) equals the first series minus twice the second, with the whole then being multiplied by the third series. If we stopped here, the target would be completely dependent on the three feature series - it would be easy to predict. However, we then add a random normal distribution to make the prediction harder.

There are a train series and a test series, with different features ("a" and "b"), and with different random normal data added. This tests whether our model can

predict new data. We fit to the "a" features using the train_target, then predict the test_target using the "b" features.

In [2]:
```
np.random.seed(0)
num_points = 10000
feature_1a = np.random.random(size = num_points) * 3
feature_2a = np.random.random(size = num_points) * 3
feature_3a = np.random.random(size = num_points) * 3
train_target = (feature_1a - 2 * feature_2a) * feature_3a + np.random.
feature_1b = np.random.random(size = num_points) * 3
feature_2b = np.random.random(size = num_points) * 3
feature_3b = np.random.random(size = num_points) * 3
test_target = (feature_1b - 2 * feature_2b) * feature_3b + np.random.n
train_df = pd.DataFrame({"f1": feature_1a, "f2": feature_2a, "f3": fea
test_df = pd.DataFrame({"f1": feature_1b, "f2": feature_2b, "f3": feat
rf = RandomForestRegressor()
rf.fit(train_df.values, train_target)
```

Out[2]:
```
▾ RandomForestRegressor
▶ Parameters
```

As expected, the RMSE is smallest when predicting the training data, larger when predicting the testing data, and largest when comparing random (shuffled) data instead of using a prediction. In the last case, there is no attempt to make a good prediction. The small RMSE when predicting the training data indicates that the model has "memorized" the training data to some degree.

In [3]:
```
print("RMSE when predicting training data:")
print(root_mean_squared_error(rf.predict(train_df.values), train_targe
print("RMSE when predicting testing data:")
print(root_mean_squared_error(rf.predict(test_df.values), test_target)
test_target_shuffle = test_target.copy()
np.random.shuffle(test_target_shuffle)
print("RMSE with random predictions:")
print(root_mean_squared_error(test_target_shuffle, test_target))
```

```
RMSE when predicting training data:
0.40447671346145025
RMSE when predicting testing data:
1.0666248639644855
RMSE with random predictions:
5.262103337484077
```

Next, we perform a Grid Search over various parameters. We've used a small number of parameters here, but you could add more. The first parameter, n_estimators, shows the number of decision trees in the random forest. The second, max_depth, is the maximum depth of each tree. By picking three

n_estimators values (10, 33, 100) and three max_depth values (2, 4, None), we allow for nine parameter combinations. For example, (n_estimators = 10, max_depth = 2) is a combination. The grid search tries all nine combinations and uses cross validation to choose the best one. This is done below.

What is cross validation?

Cross validation is a way to find out which parameter combination is the best. With 5-fold cross validation (the default) we divide the data into five equal-sized groups. Four of them are used as training data and the fifth is testing data. The score (in this case the root mean squared error) on the testing data is recorded. This is done for each split: that is, each fifth is taken out and used as testing data; then, the five root mean squared errors (from taking out the five fifths, one at a time) are averaged.

We now have root mean squared error value for each parameter combination, allowing us to choose the best combination.

In [4]:
```
np.random.seed(0)
gcv = GridSearchCV(estimator = RandomForestRegressor(), param_grid = {
                   scoring = "neg_root_mean_squared_error")
gcv.fit(train_df.values, train_target)
```

Out[4]:

```
▸ GridSearchCV
▸ best_estimator_: RandomForestRegressor
  ▸ RandomForestRegressor
```

In [5]:
```
print(gcv.best_estimator_)
print(gcv.best_params_)
print(gcv.best_score_)
```

```
RandomForestRegressor()
{'max_depth': None, 'n_estimators': 100}
-1.0851445108374207
```

In [6]:
```
gcv.cv_results_["mean_test_score"] # There are nine scores, one for ea
```

Out[6]:
```
array([-2.27296628, -2.26369708, -2.26370023, -1.56063403, -1.5273292
3,
       -1.52615891, -1.12287063, -1.09373011, -1.08514451])
```

Why does n_estimators grow like 10, 33, 100 instead of

like 1, 50, 100?

First of all, we can make it grow however we want. The question is why we chose 10, 33, 100. The idea here is that when you have no idea what is the appropriate number, you should pick guesses that grow exponentially. For example, suppose you are trying to guess the height of a particular tree. You look online and find that redwoods grow to 400 feet high, while bonsai trees may be as short as six inches. If you get three guesses, you could guess: one foot, 200 feet, 400 feet. These guesses would be linearly spaced out. However, the 200 foot guess feels wrong - it's still very tall. Most trees don't grow that tall. Instead, let's make a geometric progression:

In [7]:
```
list(map(round, [400**0, 400**0.25, 400**0.5, 400**0.75, 400*1]))
```

Out[7]:
```
[1, 4, 20, 89, 400]
```

Now the guesses are four feet, 20 feet, and 89 feet. (We'll drop the initial 1 and the final 400.) That feels more plausible: the middle value, 20 feet, is representative of many actual trees in a way that the middle 200 foot guess wasn't.

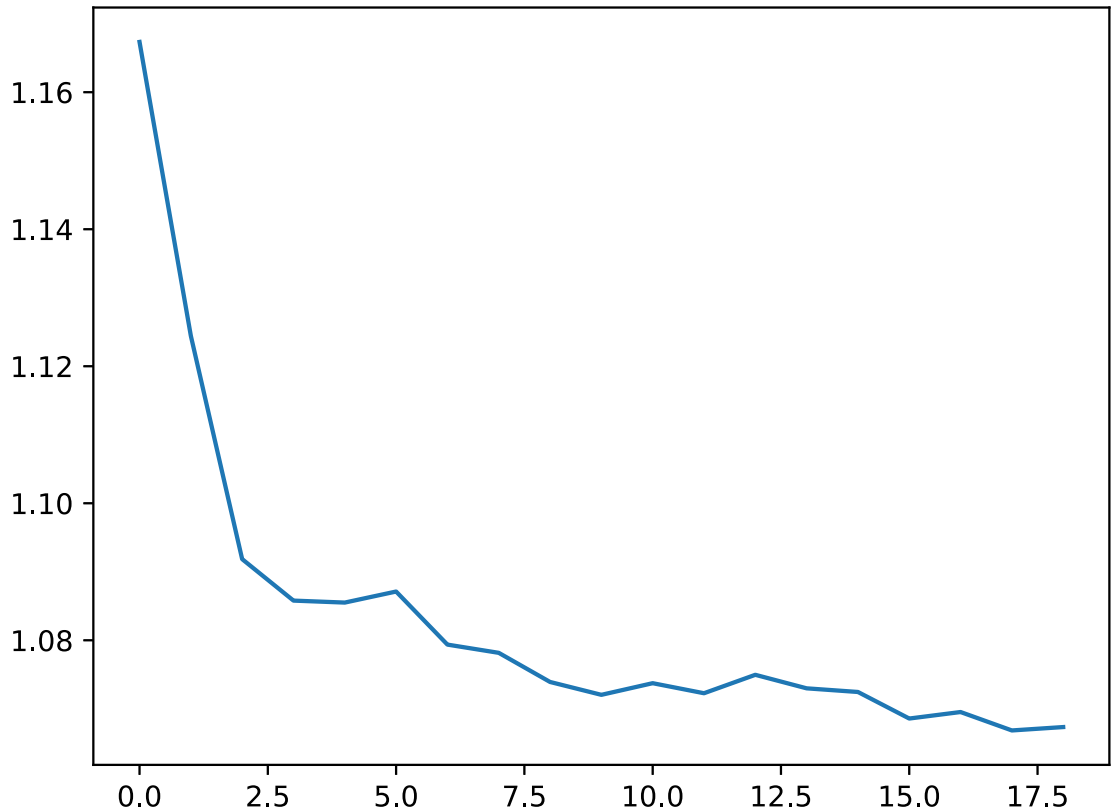
2. Weekly graph question

In words, please summarize what this graph shows.

In [8]:
```
rmse_lst = list()
for x in range(round(num_points / 20), num_points, round(num_points /
    rf.fit(train_df.values[0:x,:], train_target[0:x])
    rmse_lst.append(root_mean_squared_error(rf.predict(test_df.values)
```

In [9]:
```
import matplotlib.pyplot as plt

plt.plot(rmse_lst)
```

Out[9]:
```
[<matplotlib.lines.Line2D at 0x146eb3750>]
```



This graph shows that the root mean squared error (RMSE) of the test set predictions of a specific random forest model decreases as the size of the training set grows. The model is trained against training sets starting at 500 data points to a maximum of 10,000 in steps of 500. The lowest RMSE achieved is approximately 1.07

3. Homework

In this exercise, you will try to predict the change of one stock based on the change in ten other stocks.

In [10]:
```
def make_X_y(alpha = 0.5):
    df_hwk = pd.DataFrame()
    num_samples_hwk = 10000
    common_odd = np.random.normal(size = num_samples_hwk)
    common_even = np.random.normal(size = num_samples_hwk)
    common_three = np.random.normal(size = num_samples_hwk)
    for n in range(11):
        common = np.zeros(num_samples_hwk)
        if n % 2 == 0:
            common += common_even
        if n % 2 == 1:
            common += common_odd
        if n % 3 == 0:
```

```
            common += common_three
            common /= 2
        df_hwk["Stock_" + str(n)] = np.random.normal(size = num_sample
    X = df_hwk.values[:,:-1]
    y = df_hwk.values[:,-1]
    return X, y
```

In [11]:
```
make_X_y()
```

Out[11]:
```
(array([[ 0.62271692, -0.48070655,  1.0376677 , ..., -0.01235944,
          0.12777276, -0.29265038],
        [-2.25863339, -0.10128859, -1.61541331, ..., -1.08308356,
         -0.33863524, -0.44628776],
        [ 0.97749818, -0.54280947,  0.40276922, ..., -1.27215827,
          0.4125673 , -0.62686244],
        ...,
        [-0.17801723, -0.62199636,  0.02861698, ...,  0.74036158,
          0.94573704, -1.26127812],
        [ 0.99718494,  0.30485447, -0.9073281 , ..., -0.47594459,
          1.10664054, -0.0742904 ],
        [-0.22534661,  0.49395442,  0.03714987, ...,  0.67189159,
         -0.88998268, -1.71240927]], shape=(10000, 10)),
 array([ 2.371949  ,  0.27514928,  1.26549696, ...,  1.3194899 ,
        -0.68844046,  1.67575342], shape=(10000,)))
```

The above code creates an X with ten columns, corresponding to ten different stocks. The y is the eleventh stock. The stocks have some commonalities: the even numbered stocks are related, as are the odds. The stocks that are multiples of three are all related, too. Random forest should be able to sort through all this and decide that the eleventh stock is related to the odd numbered, non-divisible-by-three stocks, and predict accordingly. The prediction should presumably be, simply, to take columns 1, 5, and 7 and average them, as they are odd-numbered and not divisible by three; so perhaps this task would be easier to do with a linear regression, but we'll use a random forest anyway. The value "alpha" determines how important the common element is. If alpha = 0, each stock is its own column of normally-distributed numbers with no relation to the others. If alpha = 10, then the great majority of the variation in the stock values comes from the common elements.

Question:

Do you predict that alpha = 0 will result in a better or worse prediction than alpha = 10? Remember that the prediction must be performed on test data, so you'll need an X_train and y_train for training and a separate X_test and y_test for testing. I recommend using a correlation rather than a MSE or RMSE in this case, if you want to directly compare two different predictions, because the MSE and

RMSE vary with the magnitude of the data as well as with the quality of the prediction.

After you make the prediction, try it out, and comment on what you learned.

Randomized Search

Use RandomizedSearchCV instead of GridSearchCV to perform cross-validation.

Scoring

Research the scoring parameters under "regression":

https://scikit-learn.org/dev/modules/model_evaluation.html#scoring-parameter

Choose at least one that you want to learn about, use it, and research it online. Write something about what you've learned about it.

I predict alpha=0 will result in a worse prediction than alpha=10. I think this because with alpha=10, the discovered relationship between the features maps to the underlying data trend, where the even and odd numbered stocks are related.

In [12]:
```
from sklearn.model_selection import train_test_split
from sklearn.model_selection import RandomizedSearchCV
```

In [13]:
```
rf = RandomForestRegressor(random_state=42)
parameter_distribution = {
    "n_estimators": [10, 33, 100],
    "max_depth": [2, 4, None]
}
```

In [18]:
```
for alpha in [0, 10]:
    X, y = make_X_y(alpha)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_siz

    cv = RandomizedSearchCV(
        estimator=rf,
        param_distributions=parameter_distribution,
        n_iter=5,
        cv=3,
        scoring = "neg_root_mean_squared_error",
        random_state=42
    )
    cv.fit(X_train, y_train)

    best_model = cv.best_estimator_
    y_pred = best_model.predict(X_test)
    corr = np.corrcoef(y_test, y_pred)
```

```
    print(f"Correlation for Alpha={alpha}: {corr}")
```

```
Correlation for Alpha=0: [[ 1.         -0.02865293]
 [-0.02865293  1.        ]]
Correlation for Alpha=10: [[1.         0.99249147]
 [0.99249147 1.        ]]
```

https://scikit-learn.org/dev/modules/generated/sklearn.metrics.brier_score_loss.html#sklearn.metr

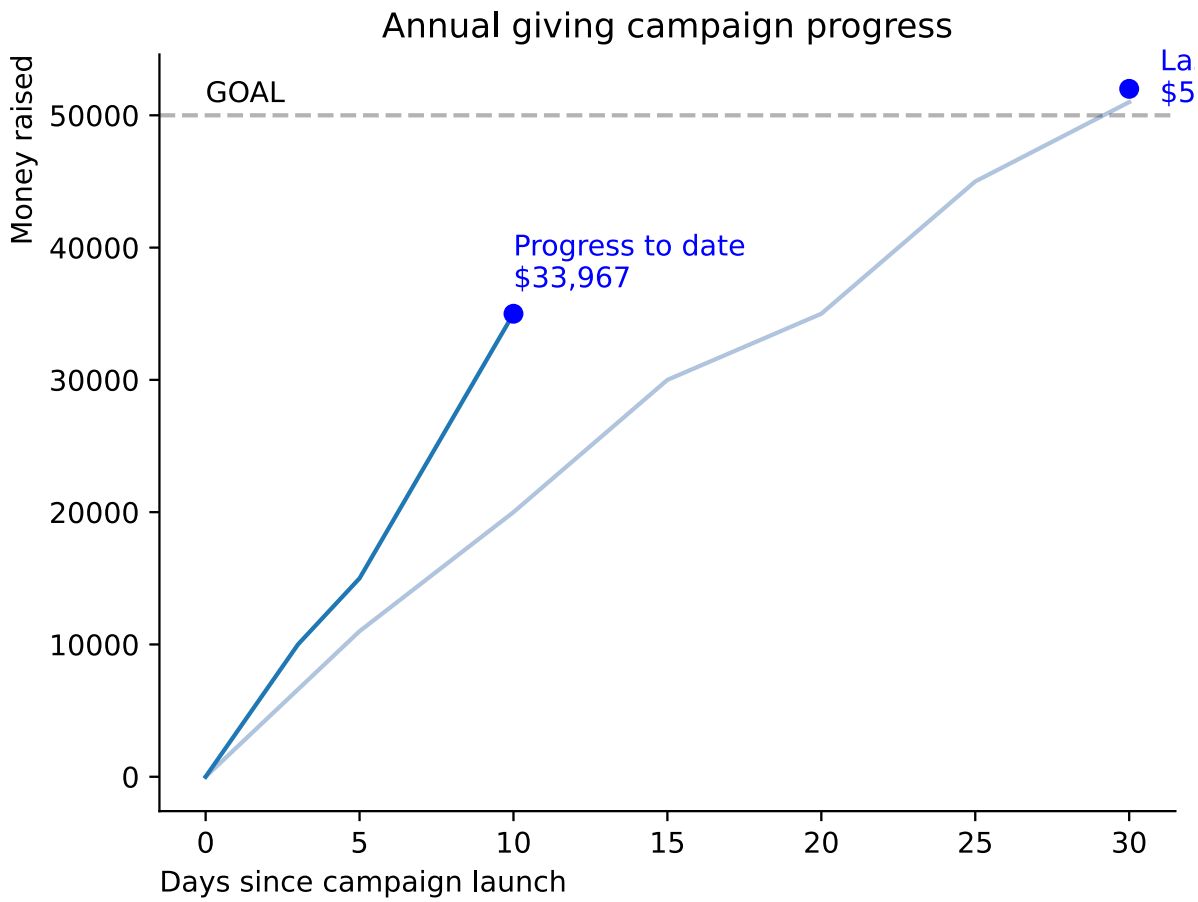
I researched the Brier score. The Brier score measures the squared differences between the predicted probabilities and outcomes. It is used for binary classification problems where we want to understand how likely an event is, rather than only the final labels.

4. Storytelling With Data plot

Reproduce any graph of your choice in chapter six (p. 151-163) of the Storytelling With Data book as best you can. You do not have to get the exact data values right, just the overall look and feel.

In [16]:
```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [17]:
```
# Figure 6.1: Line graph
x1 = [0, 5, 10, 15, 20, 25, 30]
y1 = [0, 11_000, 20_000, 30_000, 35_000, 45_000, 51_000]

x2 = [0, 3, 5, 10]
y2 = [0, 10_000, 15_000, 35_000]

plt.plot(x1,y1, color='lightsteelblue')
plt.plot(x2,y2)
plt.title("Annual giving campaign progress")

plt.text(10, 37_000, "Progress to date\n$33,967", color='Blue')
plt.plot(10, 35_000, 'bo')

plt.text(31, 51_000, "Last year\n$51,400", color='Blue')
plt.plot(30, 52_000, 'bo')

plt.ylabel("Money raised", loc='top')
plt.xlabel("Days since campaign launch",loc='left')

plt.axhline(y=50_000, color='black', linestyle='--', label='GOAL', alp
plt.text(0, 51_000, "GOAL")

sns.despine() # Defaults to removing top and right
```


Annual giving campaign progress
Money raised
GOAL
50000
40000
30000
20000
10000
0
Progress to date
$33,967
La
$5
0
5
10
15
20
25
30
Days since campaign launch

In [ ]:

In [ ]:

Week 11 - Introduction to Modeling, part 2

1. Lesson - No lesson this week

2. Weekly graph question

In [6]:
```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import root_mean_squared_error
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import make_scorer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RepeatedKFold
from sklearn.model_selection import cross_val_score
from sklearn.metrics import classification_report
```

The book names one of Vonnegut's rules as "keep it simple" and another as "have the guts to cut." Here is some data from the previous week's lesson. If you had to cut one of the two plots below, which would it be? Which seems more interesting or important? Explain. (Should "amount of training data used" or "number of estimators" be on the x-axis.)

In [7]:
```
np.random.seed(0)
num_points = 10000
feature_1a = np.random.random(size = num_points) * 3
feature_2a = np.random.random(size = num_points) * 3
feature_3a = np.random.random(size = num_points) * 3
train_target = (feature_1a - 2 * feature_2a) * feature_3a + np.random.
feature_1b = np.random.random(size = num_points) * 3
feature_2b = np.random.random(size = num_points) * 3
feature_3b = np.random.random(size = num_points) * 3
test_target = (feature_1b - 2 * feature_2b) * feature_3b + np.random.n
train_df = pd.DataFrame({"f1": feature_1a, "f2": feature_2a, "f3": fea
test_df = pd.DataFrame({"f1": feature_1b, "f2": feature_2b, "f3": feat
rf = RandomForestRegressor()
rf.fit(train_df.values, train_target)
```

Out[7]:

```
▾ RandomForestRegressor
▶ Parameters
```

In [8]:

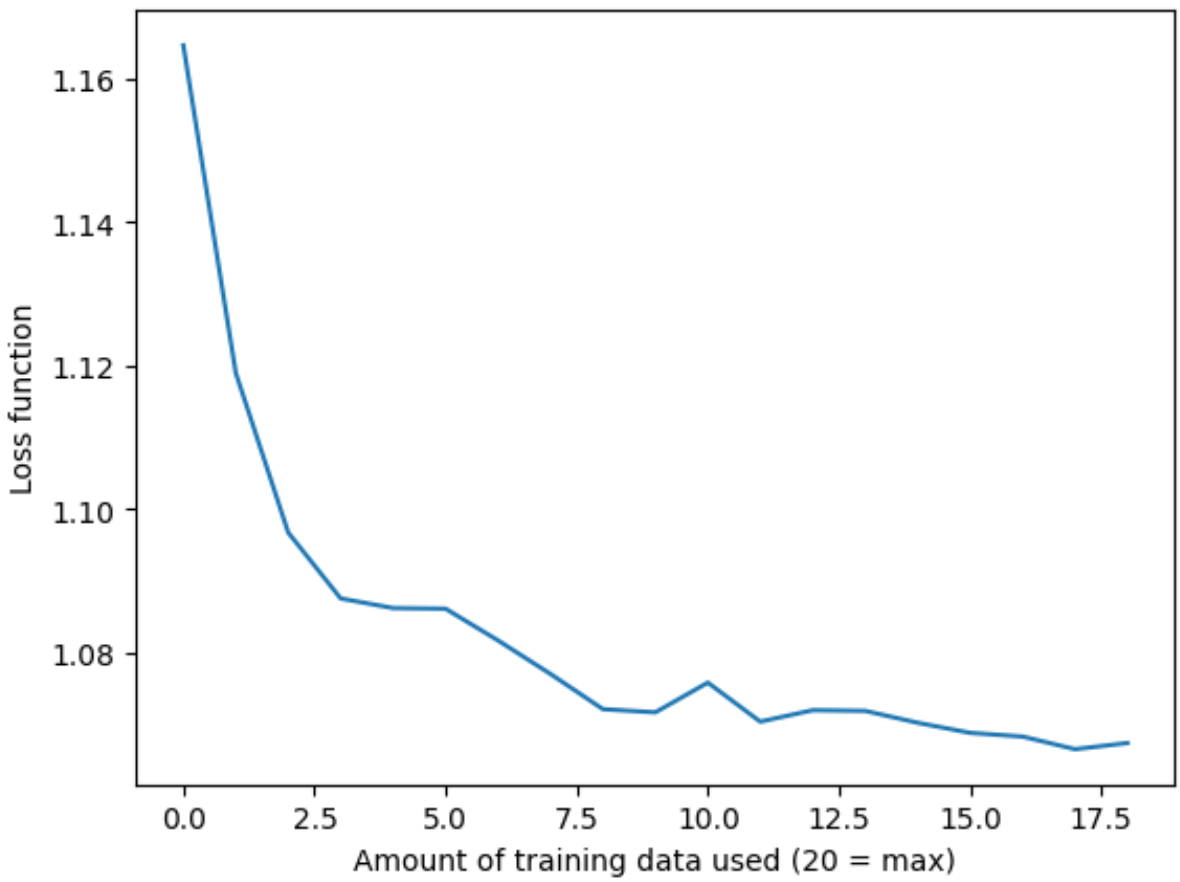
```
rmse_lst = list()
rf = RandomForestRegressor()
for x in range(round(num_points / 20), num_points, round(num_points /
    rf.fit(train_df.values[0:x,:], train_target[0:x])
    rmse_lst.append(root_mean_squared_error(rf.predict(test_df.values)

import matplotlib.pyplot as plt
```

In [9]:

```
plt.plot(rmse_lst)
plt.xlabel("Amount of training data used (20 = max)")
plt.ylabel("Loss function")
```

Out[9]:

```
Text(0, 0.5, 'Loss function')
```



In [10]:

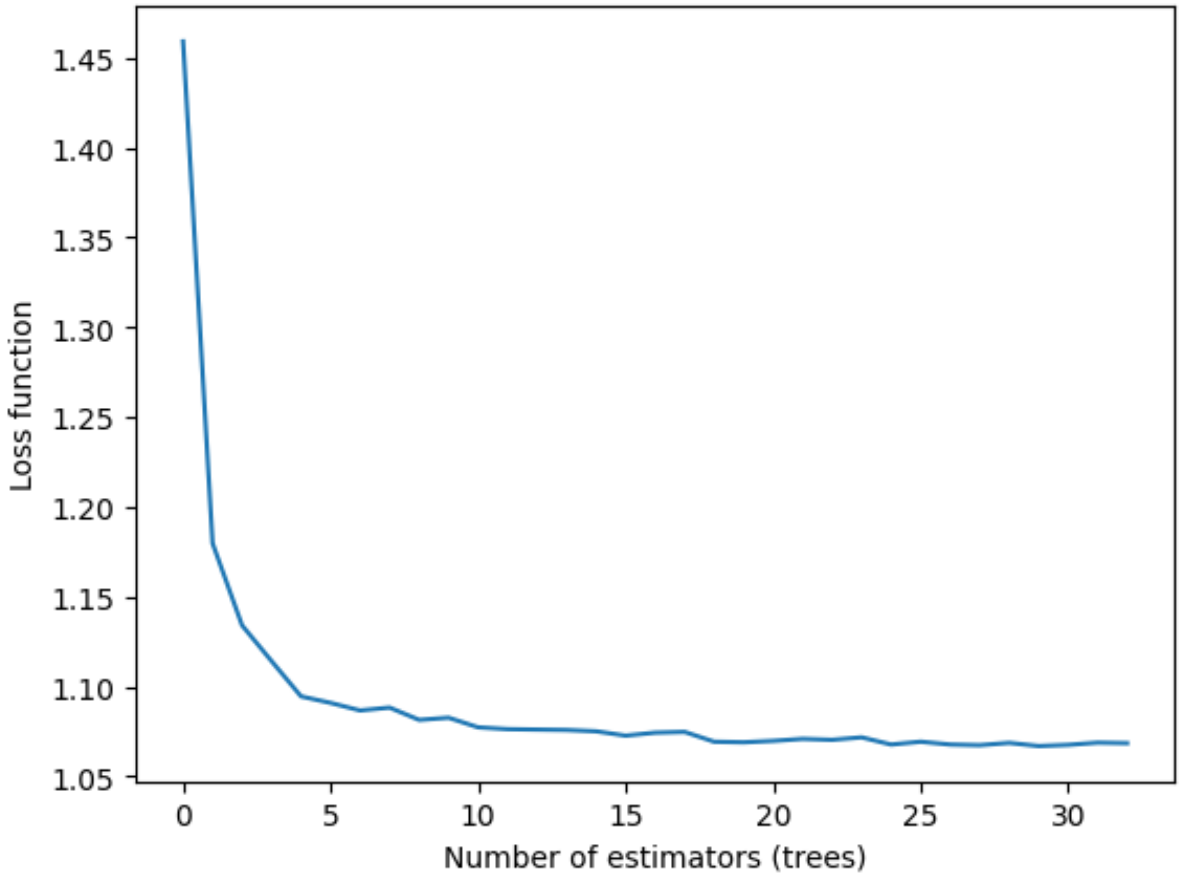
```
num_trees_lst = list()
for n_estimators in range(1, 100, 3):
    rf = RandomForestRegressor(n_estimators = n_estimators)
    rf.fit(train_df.values, train_target)
    num_trees_lst.append(root_mean_squared_error(rf.predict(test_df.va
```

In [11]:

```
plt.plot(num_trees_lst)
```

```
plt.xlabel("Number of estimators (trees)")
plt.ylabel("Loss function")
```

Out[11]: Text(0, 0.5, 'Loss function')



If I had to cut one of the plots, I would remove the Number of estimators plot. We can see past around 5 estimators, the performance does not increase much. Therefore the plot is not very informative beyond understanding that we do not need many estimators to minimize the loss.

3. Working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do:

- Implement a random forest model.
- Perform cross-validation.
- Tune hyperparameters.
- Evaluate a performance metric.

If you like, you can try other types of models, too (beyond linear regression and random forest) although you will have many opportunities to do that next semester.

In [12]:
```
df = pd.read_csv('Milestone 3 EDA/ibm_hi_small_trans_cleaned.csv')
df
```

Out[12]:

	Timestamp	From Bank	Account	To Bank	Account.1	Amount Received
0	2022/09/01 00:20	10	8000EBD30	10	8000EBD30	3697.340000
1	2022/09/01 00:20	3208	8000F4580	1	8000F5340	0.010000
2	2022/09/01 00:00	3209	8000F4670	3209	8000F4670	14675.570000
3	2022/09/01 00:02	12	8000F5030	12	8000F5030	2806.970000
4	2022/09/01 00:06	10	8000F5200	10	8000F5200	36682.970000
...	...	...	...	...	...	...
5078340	2022/09/10 23:57	54219	8148A6631	256398	8148A8711	0.154978
5078341	2022/09/10 23:35	15	8148A8671	256398	8148A8711	0.108128
5078342	2022/09/10 23:52	154365	8148A6771	256398	8148A8711	0.004988
5078343	2022/09/10 23:46	256398	8148A6311	256398	8148A8711	0.038417
5078344	2022/09/10 23:37	154518	8148A6091	256398	8148A8711	0.281983

5078345 rows × 109 columns

In [13]:
```
df.columns
```

Out[13]:
```
Index(['Timestamp', 'From Bank', 'Account', 'To Bank', 'Account.1',
       'Amount Received', 'Amount Paid', 'Is Laundering',
       'Amount_Received_USD', 'Amount_Paid_USD',
       ...
       'Payment Currency_Yuan', 'Payment Format_ACH', 'Payment Format
_Bitcoin',
       'Payment Format_Cash', 'Payment Format_Cheque',
       'Payment Format_Credit Card', 'Payment Format_Reinvestment',
       'Payment Format_Wire', 'Account_Same', 'Bank_Same'],
      dtype='object', length=109)
```

In [14]:
```
df.drop(columns=['Timestamp', 'From Bank', 'To Bank', 'Account', 'Acco
```

In [15]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5078345 entries, 0 to 5078344
Columns: 104 entries, Amount Received to Bank_Same
dtypes: float64(101), int64(3)
memory usage: 3.9 GB
```

In [16]:
```
random_state = 42
```

In [17]:
```
fraud = df[df['Is Laundering'] == 1]
non_fraud = df[df['Is Laundering'] == 0]
len(fraud), len(non_fraud)
```

Out[17]:
```
(5177, 5073168)
```

In [18]:
```
# Undersample majority class to a 1:10 ratio
non_fraud_downsampled = non_fraud.sample(n=len(fraud) * 10, random_sta
non_fraud_downsampled
```

Out[18]:

	Amount Received	Amount Paid	Is Laundering	Amount_Received_USD	Amou
4563287	1439.79	1439.79	0	3.829841e+02	3
4216770	33.09	33.09	0	3.309000e+01	3
2992877	263.34	263.34	0	3.807896e+01	3
1499016	17154983.76	17154983.76	0	1.747064e+07	1
510282	8499.22	8499.22	0	2.501320e+03	2
...	...	...	...	...	
3992741	5722.87	5722.87	0	5.722870e+03	5
44465	82208.92	82208.92	0	8.220892e+04	8
1173046	6837.09	6837.09	0	5.196872e+03	5
4125810	3261.27	3261.27	0	3.763506e+03	3
3821365	20287.92	20287.92	0	2.933633e+03	2

51770 rows × 104 columns

In [19]:

```
# Combine and shuffle
df_reduced = pd.concat([fraud, non_fraud_downsampled]).sample(frac=1).
df_reduced
```

Out[19]:

	Amount Received	Amount Paid	Is Laundering	Amount_Received_USD	Amount_Paid
0	142.21	142.21	0	141.427845	141.4
1	8.12	8.12	0	8.269408	8.26
2	234.56	234.56	0	233.269920	233.26
3	19640.00	19640.00	0	20001.376000	20001.3
4	8.38	8.38	0	8.380000	8.38
...	...	...	...	...	
56942	78922.59	78922.59	0	3906.668205	3906.66
56943	89535.13	89535.13	0	635.699423	635.69
56944	207870.51	207870.51	0	2598.381375	2598.3
56945	29.48	29.48	0	29.317860	29.3
56946	208.99	208.99	0	30.219954	30.2

56947 rows × 104 columns

In [20]:
```
X = df_reduced.drop(columns='Is Laundering')
y = df_reduced['Is Laundering']
```

In [21]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,
                                                    y,
                                                    test_size=0.2,
                                                    stratify=y,  # Mai
                                                    random_state=rando
                                                    )
```

In [47]:
```
scaler = StandardScaler()
```

In [48]:
```
X_train_scaled = scaler.fit_transform(X_train)
X_train_scaled
```

Out[48]:
```
array([[-0.01368816, -0.01278547, -0.01008913, ..., -0.18157169,
        -0.34846905, -0.38402302],
       [-0.01368897, -0.01278628, -0.01009458, ..., -0.18157169,
        -0.34846905, -0.38402302],
       [-0.01368909, -0.0127864 , -0.01009542, ..., -0.18157169,
        -0.34846905, -0.38402302],
       ...,
       [-0.01368367, -0.01278097, -0.01005863, ..., -0.18157169,
        -0.34846905, -0.38402302],
       [-0.01253343, -0.01162778, -0.01003977, ..., -0.18157169,
        -0.34846905, -0.38402302],
       [-0.01368867, -0.01278597, -0.01009256, ..., -0.18157169,
        -0.34846905, -0.38402302]], shape=(45557, 103))
```

In [49]:
```
X_test_scaled = scaler.transform(X_test)
```

In [25]:
```
# cv = RepeatedKFold(n_splits=5, n_repeats=5, random_state=random_stat
from sklearn.model_selection import StratifiedKFold


cv = StratifiedKFold(n_splits=5)
```

RF with scaled data & recall as param tuning metric

In [ ]:
```
# Parameter tuning
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score


def objective(trial):
    n_estimators = trial.suggest_int("n_estimators", 10, 200)
    max_depth = trial.suggest_int("max_depth", 2, 32, log=True)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 10)

    model = RandomForestClassifier(
        class_weight='balanced',
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        random_state=random_state
    )

    scores = cross_val_score(
        model,
        X=X_train_scaled,
        y=y_train,
        cv=cv,
        # scoring="f1",
        scoring='recall',
        n_jobs=-1,
```

```
    )
    # f1 = scores.mean()
    recall = scores.mean()

    # return f1
    return recall


study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=100)

top_5_df = study.trials_dataframe().sort_values("value", ascending=Fal
display(top_5_df)
```

```
c:\Users\caleb\anaconda3\envs\iris-fisher\lib\site-packages\tqdm\auto.p
y:21: TqdmWarning: IProgress not found. Please update jupyter and ipywi
dgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.htm
l
  from .autonotebook import tqdm as notebook_tqdm
[I 2026-04-14 14:20:48,392] A new study created in memory with name: no
-name-65efc86f-277b-40e7-beb9-528fedfb91a3
[I 2026-04-14 14:20:54,968] Trial 0 finished with value: 0.708352709451
4664 and parameters: {'n_estimators': 144, 'max_depth': 28, 'min_sample
s_split': 4}. Best is trial 0 with value: 0.7083527094514664.
[I 2026-04-14 14:21:00,310] Trial 1 finished with value: 0.770645618083
6 and parameters: {'n_estimators': 148, 'max_depth': 25, 'min_samples_s
plit': 7}. Best is trial 1 with value: 0.7706456180836.
[I 2026-04-14 14:21:02,198] Trial 2 finished with value: 0.806376636772
0844 and parameters: {'n_estimators': 116, 'max_depth': 2, 'min_samples
_split': 7}. Best is trial 2 with value: 0.8063766367720844.
[I 2026-04-14 14:21:05,192] Trial 3 finished with value: 0.826893760598
591 and parameters: {'n_estimators': 171, 'max_depth': 4, 'min_samples_
split': 5}. Best is trial 3 with value: 0.826893760598591.
[I 2026-04-14 14:21:06,681] Trial 4 finished with value: 0.831964767515
7194 and parameters: {'n_estimators': 97, 'max_depth': 5, 'min_samples_
split': 6}. Best is trial 4 with value: 0.8319647675157194.
[I 2026-04-14 14:21:07,977] Trial 5 finished with value: 0.832204273818
0567 and parameters: {'n_estimators': 74, 'max_depth': 6, 'min_samples_
split': 8}. Best is trial 5 with value: 0.8322042738180567.
[I 2026-04-14 14:21:10,086] Trial 6 finished with value: 0.788753401747
0557 and parameters: {'n_estimators': 68, 'max_depth': 23, 'min_samples
_split': 8}. Best is trial 5 with value: 0.8322042738180567.
[I 2026-04-14 14:21:12,063] Trial 7 finished with value: 0.809032184752
0148 and parameters: {'n_estimators': 191, 'max_depth': 3, 'min_samples
_split': 5}. Best is trial 5 with value: 0.8322042738180567.
[I 2026-04-14 14:21:16,910] Trial 8 finished with value: 0.817483085960
0357 and parameters: {'n_estimators': 191, 'max_depth': 17, 'min_sample
s_split': 8}. Best is trial 5 with value: 0.8322042738180567.
[I 2026-04-14 14:21:18,061] Trial 9 finished with value: 0.805168907303
4854 and parameters: {'n_estimators': 125, 'max_depth': 2, 'min_samples
_split': 2}. Best is trial 5 with value: 0.8322042738180567.
[I 2026-04-14 14:21:18,729] Trial 10 finished with value: 0.83896785021
24088 and parameters: {'n_estimators': 25, 'max_depth': 10, 'min_sample
```

```
s_split': 10}. Best is trial 10 with value: 0.8389678502124088.
[I 2026-04-14 14:21:19,210] Trial 11 finished with value: 0.82448121536
33678 and parameters: {'n_estimators': 10, 'max_depth': 9, 'min_samples
_split': 10}. Best is trial 10 with value: 0.8389678502124088.
[I 2026-04-14 14:21:20,002] Trial 12 finished with value: 0.84476378618
08943 and parameters: {'n_estimators': 31, 'max_depth': 10, 'min_sample
s_split': 10}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:20,453] Trial 13 finished with value: 0.83607017359
83638 and parameters: {'n_estimators': 13, 'max_depth': 11, 'min_sample
s_split': 10}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:21,749] Trial 14 finished with value: 0.83824612623
32243 and parameters: {'n_estimators': 50, 'max_depth': 13, 'min_sample
s_split': 10}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:22,608] Trial 15 finished with value: 0.84041654283
43328 and parameters: {'n_estimators': 39, 'max_depth': 8, 'min_samples
_split': 9}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:23,497] Trial 16 finished with value: 0.83389596918
46879 and parameters: {'n_estimators': 43, 'max_depth': 7, 'min_samples
_split': 9}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:25,784] Trial 17 finished with value: 0.82593253031
7069 and parameters: {'n_estimators': 87, 'max_depth': 16, 'min_samples
_split': 9}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:26,705] Trial 18 finished with value: 0.84017528831
08105 and parameters: {'n_estimators': 44, 'max_depth': 8, 'min_samples
_split': 9}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:27,328] Trial 19 finished with value: 0.80275490521
72749 and parameters: {'n_estimators': 36, 'max_depth': 4, 'min_samples
_split': 7}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:29,138] Trial 20 finished with value: 0.82351823686
06611 and parameters: {'n_estimators': 69, 'max_depth': 15, 'min_sample
s_split': 3}. Best is trial 12 with value: 0.8447637861808943.
[I 2026-04-14 14:21:30,275] Trial 21 finished with value: 0.84741962553
10222 and parameters: {'n_estimators': 56, 'max_depth': 8, 'min_samples
_split': 9}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:31,353] Trial 22 finished with value: 0.82520497893
39347 and parameters: {'n_estimators': 60, 'max_depth': 6, 'min_samples
_split': 9}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:31,929] Trial 23 finished with value: 0.83728081676
89374 and parameters: {'n_estimators': 20, 'max_depth': 11, 'min_sample
s_split': 8}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:32,661] Trial 24 finished with value: 0.84500445796
40215 and parameters: {'n_estimators': 31, 'max_depth': 8, 'min_samples
_split': 9}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:33,613] Trial 25 finished with value: 0.80492677866
93706 and parameters: {'n_estimators': 28, 'max_depth': 20, 'min_sample
s_split': 10}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:34,501] Trial 26 finished with value: 0.80734194623
63711 and parameters: {'n_estimators': 54, 'max_depth': 5, 'min_samples
_split': 8}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:36,417] Trial 27 finished with value: 0.83631550730
4651 and parameters: {'n_estimators': 79, 'max_depth': 13, 'min_samples
_split': 6}. Best is trial 21 with value: 0.8474196255310222.
```

```
[I 2026-04-14 14:21:37,492] Trial 28 finished with value: 0.80565170772
07275 and parameters: {'n_estimators': 92, 'max_depth': 3, 'min_samples
_split': 9}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:38,121] Trial 29 finished with value: 0.81531033839
73473 and parameters: {'n_estimators': 30, 'max_depth': 6, 'min_samples
_split': 7}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:39,996] Trial 30 finished with value: 0.76436833854
88599 and parameters: {'n_estimators': 59, 'max_depth': 31, 'min_sample
s_split': 10}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:40,916] Trial 31 finished with value: 0.83800108389
71346 and parameters: {'n_estimators': 42, 'max_depth': 8, 'min_samples
_split': 9}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:41,662] Trial 32 finished with value: 0.84645431606
67354 and parameters: {'n_estimators': 29, 'max_depth': 9, 'min_samples
_split': 9}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:42,383] Trial 33 finished with value: 0.83921056158
69185 and parameters: {'n_estimators': 20, 'max_depth': 12, 'min_sample
s_split': 7}. Best is trial 21 with value: 0.8474196255310222.
[I 2026-04-14 14:21:45,758] Trial 34 finished with value: 0.85080010256
23095 and parameters: {'n_estimators': 139, 'max_depth': 9, 'min_sample
s_split': 8}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:21:48,785] Trial 35 finished with value: 0.84790213457
80667 and parameters: {'n_estimators': 143, 'max_depth': 7, 'min_sample
s_split': 8}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:21:51,105] Trial 36 finished with value: 0.83679422853
91281 and parameters: {'n_estimators': 150, 'max_depth': 5, 'min_sample
s_split': 8}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:21:52,788] Trial 37 finished with value: 0.81458191290
36207 and parameters: {'n_estimators': 135, 'max_depth': 4, 'min_sample
s_split': 5}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:21:54,486] Trial 38 finished with value: 0.83969277926
37658 and parameters: {'n_estimators': 107, 'max_depth': 6, 'min_sample
s_split': 6}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:21:57,157] Trial 39 finished with value: 0.84910957267
64684 and parameters: {'n_estimators': 165, 'max_depth': 7, 'min_sample
s_split': 7}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:21:58,896] Trial 40 finished with value: 0.80710098308
30465 and parameters: {'n_estimators': 163, 'max_depth': 3, 'min_sample
s_split': 7}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:02,027] Trial 41 finished with value: 0.84838522636
55065 and parameters: {'n_estimators': 169, 'max_depth': 7, 'min_sample
s_split': 8}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:04,876] Trial 42 finished with value: 0.84790271731
84617 and parameters: {'n_estimators': 175, 'max_depth': 7, 'min_sample
s_split': 8}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:07,639] Trial 43 finished with value: 0.84886831815
29459 and parameters: {'n_estimators': 171, 'max_depth': 7, 'min_sample
s_split': 7}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:10,072] Trial 44 finished with value: 0.83848446705
47718 and parameters: {'n_estimators': 178, 'max_depth': 5, 'min_sample
s_split': 6}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:13,148] Trial 45 finished with value: 0.84379789397
```

```
62125 and parameters: {'n_estimators': 200, 'max_depth': 6, 'min_sample
s_split': 7}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:16,427] Trial 46 finished with value: 0.84910986404
6666 and parameters: {'n_estimators': 163, 'max_depth': 10, 'min_sample
s_split': 6}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:19,618] Trial 47 finished with value: 0.84838522636
55065 and parameters: {'n_estimators': 161, 'max_depth': 10, 'min_sampl
es_split': 5}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:24,028] Trial 48 finished with value: 0.82955513598
2471 and parameters: {'n_estimators': 185, 'max_depth': 14, 'min_sample
s_split': 4}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:27,420] Trial 49 finished with value: 0.84790271731
84617 and parameters: {'n_estimators': 159, 'max_depth': 11, 'min_sampl
es_split': 6}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:29,928] Trial 50 finished with value: 0.85080010256
23095 and parameters: {'n_estimators': 130, 'max_depth': 9, 'min_sample
s_split': 7}. Best is trial 34 with value: 0.8508001025623095.
[I 2026-04-14 14:22:32,368] Trial 51 finished with value: 0.85128319434
97492 and parameters: {'n_estimators': 124, 'max_depth': 9, 'min_sample
s_split': 7}. Best is trial 51 with value: 0.8512831943497492.
[I 2026-04-14 14:22:34,773] Trial 52 finished with value: 0.85128319434
97492 and parameters: {'n_estimators': 124, 'max_depth': 9, 'min_sample
s_split': 7}. Best is trial 51 with value: 0.8512831943497492.
[I 2026-04-14 14:22:37,142] Trial 53 finished with value: 0.85200666655
01187 and parameters: {'n_estimators': 122, 'max_depth': 9, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:39,526] Trial 54 finished with value: 0.85128202886
89593 and parameters: {'n_estimators': 123, 'max_depth': 9, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:41,884] Trial 55 finished with value: 0.85152386613
28766 and parameters: {'n_estimators': 121, 'max_depth': 9, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:44,878] Trial 56 finished with value: 0.81024020559
08113 and parameters: {'n_estimators': 109, 'max_depth': 18, 'min_sampl
es_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:47,516] Trial 57 finished with value: 0.84210911231
15563 and parameters: {'n_estimators': 119, 'max_depth': 12, 'min_sampl
es_split': 5}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:49,818] Trial 58 finished with value: 0.85128202886
89593 and parameters: {'n_estimators': 119, 'max_depth': 9, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:52,442] Trial 59 finished with value: 0.84669586196
04552 and parameters: {'n_estimators': 123, 'max_depth': 11, 'min_sampl
es_split': 5}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:54,886] Trial 60 finished with value: 0.83052073681
69555 and parameters: {'n_estimators': 100, 'max_depth': 14, 'min_sampl
es_split': 4}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:57,091] Trial 61 finished with value: 0.85007546488
11502 and parameters: {'n_estimators': 114, 'max_depth': 9, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:22:59,663] Trial 62 finished with value: 0.84959208172
3513 and parameters: {'n_estimators': 137, 'max_depth': 9, 'min_samples
```

```
_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:02,316] Trial 63 finished with value: 0.84838493499
5309 and parameters: {'n_estimators': 128, 'max_depth': 10, 'min_sample
s_split': 5}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:04,953] Trial 64 finished with value: 0.84935111857
01882 and parameters: {'n_estimators': 152, 'max_depth': 8, 'min_sample
s_split': 7}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:07,478] Trial 65 finished with value: 0.84307616999
7028 and parameters: {'n_estimators': 115, 'max_depth': 12, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:10,504] Trial 66 finished with value: 0.84959208172
3513 and parameters: {'n_estimators': 137, 'max_depth': 9, 'min_samples
_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:12,836] Trial 67 finished with value: 0.85007546488
11502 and parameters: {'n_estimators': 99, 'max_depth': 8, 'min_samples
_split': 5}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:15,744] Trial 68 finished with value: 0.84911015541
68633 and parameters: {'n_estimators': 122, 'max_depth': 10, 'min_sampl
es_split': 7}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:19,677] Trial 69 finished with value: 0.83848796349
71417 and parameters: {'n_estimators': 144, 'max_depth': 13, 'min_sampl
es_split': 7}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:22,175] Trial 70 finished with value: 0.83896785021
24089 and parameters: {'n_estimators': 132, 'max_depth': 6, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:25,485] Trial 71 finished with value: 0.85080010256
23095 and parameters: {'n_estimators': 131, 'max_depth': 9, 'min_sample
s_split': 7}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:27,923] Trial 72 finished with value: 0.84886802678
27485 and parameters: {'n_estimators': 110, 'max_depth': 8, 'min_sample
s_split': 8}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:31,004] Trial 73 finished with value: 0.85031701077
487 and parameters: {'n_estimators': 126, 'max_depth': 9, 'min_samples_
split': 7}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:34,817] Trial 74 finished with value: 0.84669557059
02578 and parameters: {'n_estimators': 140, 'max_depth': 11, 'min_sampl
es_split': 5}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:36,111] Trial 75 finished with value: 0.80516890730
34854 and parameters: {'n_estimators': 118, 'max_depth': 2, 'min_sample
s_split': 7}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:38,598] Trial 76 finished with value: 0.84935082719
99908 and parameters: {'n_estimators': 104, 'max_depth': 8, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:42,519] Trial 77 finished with value: 0.84766058868
43471 and parameters: {'n_estimators': 155, 'max_depth': 10, 'min_sampl
es_split': 2}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:45,049] Trial 78 finished with value: 0.84645577291
7723 and parameters: {'n_estimators': 93, 'max_depth': 12, 'min_samples
_split': 8}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:47,642] Trial 79 finished with value: 0.84814338910
15892 and parameters: {'n_estimators': 129, 'max_depth': 7, 'min_sample
s_split': 7}. Best is trial 53 with value: 0.8520066665501187.
```

```
[I 2026-04-14 14:23:52,012] Trial 80 finished with value: 0.82714084252
60631 and parameters: {'n_estimators': 146, 'max_depth': 15, 'min_sampl
es_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:55,058] Trial 81 finished with value: 0.85007546488
11502 and parameters: {'n_estimators': 133, 'max_depth': 9, 'min_sample
s_split': 7}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:23:57,841] Trial 82 finished with value: 0.85152386613
28766 and parameters: {'n_estimators': 121, 'max_depth': 9, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:24:00,389] Trial 83 finished with value: 0.85031701077
487 and parameters: {'n_estimators': 113, 'max_depth': 8, 'min_samples_
split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:24:03,247] Trial 84 finished with value: 0.84935140994
03857 and parameters: {'n_estimators': 122, 'max_depth': 10, 'min_sampl
es_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:24:06,037] Trial 85 finished with value: 0.84886773541
2551 and parameters: {'n_estimators': 140, 'max_depth': 7, 'min_samples
_split': 7}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:24:08,837] Trial 86 finished with value: 0.84790300868
86592 and parameters: {'n_estimators': 105, 'max_depth': 11, 'min_sampl
es_split': 8}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:24:11,079] Trial 87 finished with value: 0.84017470557
04154 and parameters: {'n_estimators': 119, 'max_depth': 6, 'min_sample
s_split': 6}. Best is trial 53 with value: 0.8520066665501187.
[I 2026-04-14 14:24:13,947] Trial 88 finished with value: 0.85249004970
77559 and parameters: {'n_estimators': 124, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:18,147] Trial 89 finished with value: 0.73949960082
28294 and parameters: {'n_estimators': 112, 'max_depth': 25, 'min_sampl
es_split': 4}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:20,347] Trial 90 finished with value: 0.84669557059
02578 and parameters: {'n_estimators': 82, 'max_depth': 11, 'min_sample
s_split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:23,184] Trial 91 finished with value: 0.85224850381
40359 and parameters: {'n_estimators': 126, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:25,952] Trial 92 finished with value: 0.85031701077
487 and parameters: {'n_estimators': 126, 'max_depth': 8, 'min_samples_
split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:28,674] Trial 93 finished with value: 0.85176570339
67938 and parameters: {'n_estimators': 118, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:31,645] Trial 94 finished with value: 0.85031730214
50674 and parameters: {'n_estimators': 120, 'max_depth': 10, 'min_sampl
es_split': 4}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:34,170] Trial 95 finished with value: 0.84935111857
01882 and parameters: {'n_estimators': 125, 'max_depth': 7, 'min_sample
s_split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:36,727] Trial 96 finished with value: 0.85200754066
0711 and parameters: {'n_estimators': 103, 'max_depth': 9, 'min_samples
_split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:39,584] Trial 97 finished with value: 0.84283345862
```

```
25181 and parameters: {'n_estimators': 101, 'max_depth': 12, 'min_sampl
es_split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:41,903] Trial 98 finished with value: 0.85007575625
13476 and parameters: {'n_estimators': 108, 'max_depth': 8, 'min_sample
s_split': 5}. Best is trial 88 with value: 0.8524900497077559.
[I 2026-04-14 14:24:44,238] Trial 99 finished with value: 0.84838551773
57039 and parameters: {'n_estimators': 94, 'max_depth': 10, 'min_sample
s_split': 4}. Best is trial 88 with value: 0.8524900497077559.
```

	number	value	datetime_start	datetime_complete	duration	pa
88	88	0.852490	2026-04-14 14:24:11.079564	2026-04-14 14:24:13.947634	0 days 00:00:02.868070	
91	91	0.852249	2026-04-14 14:24:20.348595	2026-04-14 14:24:23.184013	0 days 00:00:02.835418	
96	96	0.852008	2026-04-14 14:24:34.171165	2026-04-14 14:24:36.727563	0 days 00:00:02.556398	
53	53	0.852007	2026-04-14 14:22:34.775041	2026-04-14 14:22:37.142433	0 days 00:00:02.367392	
93	93	0.851766	2026-04-14 14:24:25.953199	2026-04-14 14:24:28.674108	0 days 00:00:02.720909	

In [43]:
```
# Train using optimal parameters
model = RandomForestClassifier(
    class_weight='balanced',
    n_estimators=top_5_df.head(1)['params_n_estimators'].item(),
    max_depth=top_5_df.head(1)['params_max_depth'].item(),
    min_samples_split=top_5_df.head(1)['params_min_samples_split'].ite
    random_state=random_state
)
model.fit(X_train_scaled, y_train)
```

Out[43]:

```
▾ RandomForestClassifier
▶ Parameters
```

In [50]:
```
y_pred = model.predict(X_test_scaled)
```

In [51]:
```
print(classification_report(y_true=y_test, y_pred=y_pred))
```

```
              precision    recall  f1-score   support

           0       0.98      0.93      0.96     10355
           1       0.55      0.85      0.67      1035

    accuracy                           0.92     11390
   macro avg       0.77      0.89      0.81     11390
weighted avg       0.94      0.92      0.93     11390
```

RF With unscaled Data and recall as param tuning metric

In [58]:
```
def objective(trial):
    n_estimators = trial.suggest_int("n_estimators", 10, 200)
    max_depth = trial.suggest_int("max_depth", 2, 32, log=True)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 10)

    model = RandomForestClassifier(
        class_weight='balanced',
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        random_state=random_state
    )

    scores = cross_val_score(
        model,
        X=X_train,
        y=y_train,
        cv=cv,
        scoring='recall',
        n_jobs=-1,
    )
    recall = scores.mean()

    return recall


study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=100)

top_5_unscaled_df = study.trials_dataframe().sort_values("value", asce
display(top_5_unscaled_df)
```

```
[I 2026-04-14 14:51:01,029] A new study created in memory with name: no
-name-81346362-09b7-423e-a1de-c904b184b1e7
[I 2026-04-14 14:51:01,605] Trial 0 finished with value: 0.816273025529
8568 and parameters: {'n_estimators': 75, 'max_depth': 2, 'min_samples_
split': 6}. Best is trial 0 with value: 0.8162730255298568.
[I 2026-04-14 14:51:02,244] Trial 1 finished with value: 0.804203306469
0012 and parameters: {'n_estimators': 61, 'max_depth': 3, 'min_samples_
split': 9}. Best is trial 0 with value: 0.8162730255298568.
```

```
[I 2026-04-14 14:51:04,307] Trial 2 finished with value: 0.823276690966
9411 and parameters: {'n_estimators': 71, 'max_depth': 17, 'min_samples
_split': 10}. Best is trial 2 with value: 0.8232766909669411.
[I 2026-04-14 14:51:05,011] Trial 3 finished with value: 0.804686106886
2432 and parameters: {'n_estimators': 49, 'max_depth': 5, 'min_samples_
split': 5}. Best is trial 2 with value: 0.8232766909669411.
[I 2026-04-14 14:51:05,320] Trial 4 finished with value: 0.839936073378
6704 and parameters: {'n_estimators': 14, 'max_depth': 2, 'min_samples_
split': 7}. Best is trial 4 with value: 0.8399360733786704.
[I 2026-04-14 14:51:06,147] Trial 5 finished with value: 0.809514985169
257 and parameters: {'n_estimators': 75, 'max_depth': 4, 'min_samples_s
plit': 4}. Best is trial 4 with value: 0.8399360733786704.
[I 2026-04-14 14:51:06,724] Trial 6 finished with value: 0.816997371840
8187 and parameters: {'n_estimators': 70, 'max_depth': 2, 'min_samples_
split': 10}. Best is trial 4 with value: 0.8399360733786704.
[I 2026-04-14 14:51:07,239] Trial 7 finished with value: 0.808549384334
7727 and parameters: {'n_estimators': 59, 'max_depth': 2, 'min_samples_
split': 9}. Best is trial 4 with value: 0.8399360733786704.
[I 2026-04-14 14:51:08,004] Trial 8 finished with value: 0.835586499070
5291 and parameters: {'n_estimators': 44, 'max_depth': 7, 'min_samples_
split': 2}. Best is trial 4 with value: 0.8399360733786704.
[I 2026-04-14 14:51:08,785] Trial 9 finished with value: 0.816277396082
819 and parameters: {'n_estimators': 22, 'max_depth': 17, 'min_samples_
split': 7}. Best is trial 4 with value: 0.8399360733786704.
[I 2026-04-14 14:51:11,434] Trial 10 finished with value: 0.85128232023
91566 and parameters: {'n_estimators': 157, 'max_depth': 9, 'min_sample
s_split': 7}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:14,570] Trial 11 finished with value: 0.84814338910
15892 and parameters: {'n_estimators': 176, 'max_depth': 10, 'min_sampl
es_split': 7}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:18,043] Trial 12 finished with value: 0.84597064153
89009 and parameters: {'n_estimators': 176, 'max_depth': 11, 'min_sampl
es_split': 7}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:20,793] Trial 13 finished with value: 0.85079981119
2112 and parameters: {'n_estimators': 167, 'max_depth': 9, 'min_samples
_split': 8}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:24,885] Trial 14 finished with value: 0.78826972721
92212 and parameters: {'n_estimators': 138, 'max_depth': 23, 'min_sampl
es_split': 8}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:27,039] Trial 15 finished with value: 0.85055855666
85896 and parameters: {'n_estimators': 139, 'max_depth': 8, 'min_sample
s_split': 4}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:30,360] Trial 16 finished with value: 0.83269290163
92487 and parameters: {'n_estimators': 144, 'max_depth': 14, 'min_sampl
es_split': 8}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:32,461] Trial 17 finished with value: 0.83751915759
0485 and parameters: {'n_estimators': 198, 'max_depth': 5, 'min_samples
_split': 5}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:35,859] Trial 18 finished with value: 0.75929733163
17313 and parameters: {'n_estimators': 109, 'max_depth': 28, 'min_sampl
es_split': 8}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:37,892] Trial 19 finished with value: 0.84790242594
```

```
82644 and parameters: {'n_estimators': 108, 'max_depth': 10, 'min_sampl
es_split': 6}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:39,905] Trial 20 finished with value: 0.84428011165
30596 and parameters: {'n_estimators': 162, 'max_depth': 6, 'min_sample
s_split': 9}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:41,999] Trial 21 finished with value: 0.84862618951
88314 and parameters: {'n_estimators': 135, 'max_depth': 8, 'min_sample
s_split': 3}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:44,372] Trial 22 finished with value: 0.85080010256
23095 and parameters: {'n_estimators': 158, 'max_depth': 8, 'min_sample
s_split': 4}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:47,562] Trial 23 finished with value: 0.84235036683
50787 and parameters: {'n_estimators': 156, 'max_depth': 12, 'min_sampl
es_split': 5}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:49,367] Trial 24 finished with value: 0.83293095109
05987 and parameters: {'n_estimators': 199, 'max_depth': 4, 'min_sample
s_split': 3}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:51,362] Trial 25 finished with value: 0.84935111857
01881 and parameters: {'n_estimators': 124, 'max_depth': 8, 'min_sample
s_split': 6}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:55,323] Trial 26 finished with value: 0.82979668187
61908 and parameters: {'n_estimators': 173, 'max_depth': 14, 'min_sampl
es_split': 4}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:51:58,100] Trial 27 finished with value: 0.76026526342
77955 and parameters: {'n_estimators': 92, 'max_depth': 21, 'min_sample
s_split': 2}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:52:00,132] Trial 28 finished with value: 0.84452224028
71744 and parameters: {'n_estimators': 163, 'max_depth': 6, 'min_sample
s_split': 8}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:52:01,552] Trial 29 finished with value: 0.80903218475
20148 and parameters: {'n_estimators': 185, 'max_depth': 3, 'min_sample
s_split': 6}. Best is trial 10 with value: 0.8512823202391566.
[I 2026-04-14 14:52:03,643] Trial 30 finished with value: 0.85128232023
91567 and parameters: {'n_estimators': 120, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 30 with value: 0.8512823202391567.
[I 2026-04-14 14:52:05,796] Trial 31 finished with value: 0.85152357476
26789 and parameters: {'n_estimators': 122, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:08,352] Trial 32 finished with value: 0.84307558725
6633 and parameters: {'n_estimators': 120, 'max_depth': 12, 'min_sample
s_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:10,227] Trial 33 finished with value: 0.84210794683
07664 and parameters: {'n_estimators': 149, 'max_depth': 6, 'min_sample
s_split': 3}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:12,682] Trial 34 finished with value: 0.81796617774
74753 and parameters: {'n_estimators': 90, 'max_depth': 16, 'min_sample
s_split': 4}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:14,456] Trial 35 finished with value: 0.84910928130
62708 and parameters: {'n_estimators': 125, 'max_depth': 7, 'min_sample
s_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:15,613] Trial 36 finished with value: 0.82761868964
99478 and parameters: {'n_estimators': 98, 'max_depth': 5, 'min_samples
```

```
_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:18,140] Trial 37 finished with value: 0.85104106571
56344 and parameters: {'n_estimators': 151, 'max_depth': 9, 'min_sample
s_split': 6}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:19,267] Trial 38 finished with value: 0.81216936766
83975 and parameters: {'n_estimators': 117, 'max_depth': 4, 'min_sample
s_split': 6}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:22,512] Trial 39 finished with value: 0.83752265403
28548 and parameters: {'n_estimators': 149, 'max_depth': 13, 'min_sampl
es_split': 6}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:24,972] Trial 40 finished with value: 0.84790271731
84617 and parameters: {'n_estimators': 129, 'max_depth': 10, 'min_sampl
es_split': 7}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:27,560] Trial 41 finished with value: 0.84983304487
68379 and parameters: {'n_estimators': 155, 'max_depth': 9, 'min_sample
s_split': 4}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:30,273] Trial 42 finished with value: 0.84959208172
35131 and parameters: {'n_estimators': 189, 'max_depth': 7, 'min_sample
s_split': 6}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:32,646] Trial 43 finished with value: 0.84886860952
31437 and parameters: {'n_estimators': 152, 'max_depth': 8, 'min_sample
s_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:35,820] Trial 44 finished with value: 0.84524746070
87289 and parameters: {'n_estimators': 134, 'max_depth': 11, 'min_sampl
es_split': 7}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:38,185] Trial 45 finished with value: 0.85128232023
91567 and parameters: {'n_estimators': 115, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:40,384] Trial 46 finished with value: 0.82327756507
75336 and parameters: {'n_estimators': 82, 'max_depth': 16, 'min_sample
s_split': 6}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:42,362] Trial 47 finished with value: 0.85055768255
79974 and parameters: {'n_estimators': 112, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:43,767] Trial 48 finished with value: 0.84114147188
56897 and parameters: {'n_estimators': 102, 'max_depth': 6, 'min_sample
s_split': 7}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:46,451] Trial 49 finished with value: 0.84935140994
03857 and parameters: {'n_estimators': 141, 'max_depth': 10, 'min_sampl
es_split': 6}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:47,965] Trial 50 finished with value: 0.84404060535
07223 and parameters: {'n_estimators': 64, 'max_depth': 12, 'min_sample
s_split': 10}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:50,294] Trial 51 finished with value: 0.84959179035
33154 and parameters: {'n_estimators': 169, 'max_depth': 7, 'min_sample
s_split': 4}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:52,514] Trial 52 finished with value: 0.84983304487
68379 and parameters: {'n_estimators': 130, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:54,384] Trial 53 finished with value: 0.84959237309
37104 and parameters: {'n_estimators': 117, 'max_depth': 8, 'min_sample
s_split': 4}. Best is trial 31 with value: 0.8515235747626789.
```

```
[I 2026-04-14 14:52:57,868] Trial 54 finished with value: 0.84548784112
16586 and parameters: {'n_estimators': 182, 'max_depth': 11, 'min_sampl
es_split': 3}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:52:59,739] Trial 55 finished with value: 0.84186610956
6849 and parameters: {'n_estimators': 145, 'max_depth': 6, 'min_samples
_split': 7}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:02,436] Trial 56 finished with value: 0.84983333624
70353 and parameters: {'n_estimators': 161, 'max_depth': 9, 'min_sample
s_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:05,355] Trial 57 finished with value: 0.80227210480
00327 and parameters: {'n_estimators': 103, 'max_depth': 18, 'min_sampl
es_split': 4}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:08,065] Trial 58 finished with value: 0.83221068396
24016 and parameters: {'n_estimators': 113, 'max_depth': 14, 'min_sampl
es_split': 6}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:10,017] Trial 59 finished with value: 0.84886773541
2551 and parameters: {'n_estimators': 138, 'max_depth': 7, 'min_samples
_split': 5}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:10,501] Trial 60 finished with value: 0.80275519658
74723 and parameters: {'n_estimators': 26, 'max_depth': 5, 'min_samples
_split': 4}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:13,558] Trial 61 finished with value: 0.84910928130
62708 and parameters: {'n_estimators': 167, 'max_depth': 10, 'min_sampl
es_split': 8}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:16,309] Trial 62 finished with value: 0.85055797392
81947 and parameters: {'n_estimators': 178, 'max_depth': 8, 'min_sample
s_split': 9}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:19,486] Trial 63 finished with value: 0.84645373332
63404 and parameters: {'n_estimators': 159, 'max_depth': 11, 'min_sampl
es_split': 7}. Best is trial 31 with value: 0.8515235747626789.
[I 2026-04-14 14:53:22,685] Trial 64 finished with value: 0.85176570339
67938 and parameters: {'n_estimators': 193, 'max_depth': 9, 'min_sample
s_split': 8}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:25,517] Trial 65 finished with value: 0.85031613666
42775 and parameters: {'n_estimators': 189, 'max_depth': 8, 'min_sample
s_split': 7}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:27,774] Trial 66 finished with value: 0.85079981119
2112 and parameters: {'n_estimators': 124, 'max_depth': 9, 'min_samples
_split': 9}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:30,450] Trial 67 finished with value: 0.84959179035
33154 and parameters: {'n_estimators': 194, 'max_depth': 7, 'min_sample
s_split': 8}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:33,567] Trial 68 finished with value: 0.84162776874
53017 and parameters: {'n_estimators': 146, 'max_depth': 12, 'min_sampl
es_split': 6}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:34,863] Trial 69 finished with value: 0.83872630431
86891 and parameters: {'n_estimators': 95, 'max_depth': 6, 'min_samples
_split': 5}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:38,012] Trial 70 finished with value: 0.83221097533
25991 and parameters: {'n_estimators': 132, 'max_depth': 14, 'min_sampl
es_split': 3}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:40,987] Trial 71 finished with value: 0.85176570339
```

```
67938 and parameters: {'n_estimators': 175, 'max_depth': 9, 'min_sample
s_split': 8}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:44,107] Trial 72 finished with value: 0.84935053582
9793 and parameters: {'n_estimators': 173, 'max_depth': 10, 'min_sample
s_split': 8}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:47,049] Trial 73 finished with value: 0.85080010256
23095 and parameters: {'n_estimators': 178, 'max_depth': 9, 'min_sample
s_split': 9}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:50,539] Trial 74 finished with value: 0.84476436892
12893 and parameters: {'n_estimators': 184, 'max_depth': 11, 'min_sampl
es_split': 7}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:52,803] Trial 75 finished with value: 0.84935053582
97933 and parameters: {'n_estimators': 166, 'max_depth': 7, 'min_sample
s_split': 8}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:56,898] Trial 76 finished with value: 0.83703956224
54152 and parameters: {'n_estimators': 193, 'max_depth': 13, 'min_sampl
es_split': 5}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:53:58,319] Trial 77 finished with value: 0.84838522636
55065 and parameters: {'n_estimators': 84, 'max_depth': 8, 'min_samples
_split': 8}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:54:01,172] Trial 78 finished with value: 0.84790242594
82643 and parameters: {'n_estimators': 157, 'max_depth': 10, 'min_sampl
es_split': 7}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:54:02,130] Trial 79 finished with value: 0.80951498516
9257 and parameters: {'n_estimators': 152, 'max_depth': 2, 'min_samples
_split': 5}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:54:07,400] Trial 80 finished with value: 0.71390622541
56396 and parameters: {'n_estimators': 172, 'max_depth': 32, 'min_sampl
es_split': 6}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:54:10,361] Trial 81 finished with value: 0.85080010256
23095 and parameters: {'n_estimators': 178, 'max_depth': 9, 'min_sample
s_split': 9}. Best is trial 64 with value: 0.8517657033967938.
[I 2026-04-14 14:54:13,319] Trial 82 finished with value: 0.85200695792
0316 and parameters: {'n_estimators': 181, 'max_depth': 9, 'min_samples
_split': 10}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:15,026] Trial 83 finished with value: 0.84935140994
03857 and parameters: {'n_estimators': 108, 'max_depth': 8, 'min_sample
s_split': 10}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:17,680] Trial 84 finished with value: 0.84959149898
31179 and parameters: {'n_estimators': 196, 'max_depth': 7, 'min_sample
s_split': 10}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:20,992] Trial 85 finished with value: 0.84910928130
62708 and parameters: {'n_estimators': 188, 'max_depth': 10, 'min_sampl
es_split': 6}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:23,391] Trial 86 finished with value: 0.84766058868
43471 and parameters: {'n_estimators': 122, 'max_depth': 11, 'min_sampl
es_split': 9}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:26,949] Trial 87 finished with value: 0.83703985361
56127 and parameters: {'n_estimators': 165, 'max_depth': 13, 'min_sampl
es_split': 5}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:30,241] Trial 88 finished with value: 0.84935053582
97933 and parameters: {'n_estimators': 200, 'max_depth': 9, 'min_sample
```

```
s_split': 4}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:32,454] Trial 89 finished with value: 0.84983391898
74302 and parameters: {'n_estimators': 141, 'max_depth': 8, 'min_sample
s_split': 10}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:33,887] Trial 90 finished with value: 0.82858545596
5222 and parameters: {'n_estimators': 128, 'max_depth': 5, 'min_samples
_split': 6}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:37,050] Trial 91 finished with value: 0.85079951982
19145 and parameters: {'n_estimators': 180, 'max_depth': 10, 'min_sampl
es_split': 9}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:39,923] Trial 92 finished with value: 0.84983391898
74302 and parameters: {'n_estimators': 171, 'max_depth': 9, 'min_sample
s_split': 9}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:42,850] Trial 93 finished with value: 0.85176570339
67938 and parameters: {'n_estimators': 175, 'max_depth': 9, 'min_sample
s_split': 8}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:45,375] Trial 94 finished with value: 0.84983304487
68379 and parameters: {'n_estimators': 185, 'max_depth': 7, 'min_sample
s_split': 8}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:48,481] Trial 95 finished with value: 0.84524658659
81365 and parameters: {'n_estimators': 162, 'max_depth': 11, 'min_sampl
es_split': 8}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:50,808] Trial 96 finished with value: 0.84959208172
3513 and parameters: {'n_estimators': 153, 'max_depth': 8, 'min_samples
_split': 7}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:53,633] Trial 97 finished with value: 0.85176570339
67938 and parameters: {'n_estimators': 175, 'max_depth': 9, 'min_sample
s_split': 8}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:54:57,528] Trial 98 finished with value: 0.84307529588
64355 and parameters: {'n_estimators': 191, 'max_depth': 12, 'min_sampl
es_split': 8}. Best is trial 82 with value: 0.852006957920316.
[I 2026-04-14 14:55:00,618] Trial 99 finished with value: 0.84790184320
78693 and parameters: {'n_estimators': 175, 'max_depth': 10, 'min_sampl
es_split': 7}. Best is trial 82 with value: 0.852006957920316.
```

	number	value	datetime_start	datetime_complete	duration	pa
82	82	0.852007	2026-04-14 14:54:10.362024	2026-04-14 14:54:13.319674	0 days 00:00:02.957650	
93	93	0.851766	2026-04-14 14:54:39.923495	2026-04-14 14:54:42.850984	0 days 00:00:02.927489	
64	64	0.851766	2026-04-14 14:53:19.486939	2026-04-14 14:53:22.685829	0 days 00:00:03.198890	
71	71	0.851766	2026-04-14 14:53:38.013142	2026-04-14 14:53:40.987306	0 days 00:00:02.974164	
97	97	0.851766	2026-04-14 14:54:50.809066	2026-04-14 14:54:53.633662	0 days 00:00:02.824596	

In [59]:
```
# Train using optimal parameters
model = RandomForestClassifier(
```

```
    class_weight='balanced',
    n_estimators=top_5_unscaled_df.head(1)['params_n_estimators'].item
    max_depth=top_5_unscaled_df.head(1)['params_max_depth'].item(),
    min_samples_split=top_5_unscaled_df.head(1)['params_min_samples_sp
    random_state=random_state
)
model.fit(X_train, y_train)
```

Out[59]:

```
▾ RandomForestClassifier
▶ Parameters
```

In [60]:

```
y_pred = model.predict(X_test)
```

In [61]:

```
print(classification_report(y_true=y_test, y_pred=y_pred))
```

```
              precision    recall  f1-score   support

           0       0.98      0.93      0.96     10355
           1       0.55      0.84      0.67      1035

    accuracy                           0.92     11390
   macro avg       0.77      0.89      0.81     11390
weighted avg       0.94      0.92      0.93     11390
```

In [62]:

```
recall_model_report_df = classification_report(y_test, y_pred, output_
```

No scaling, f1 as eval metric

In [ ]:

```
# Parameter tuning
import optuna
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score


def objective(trial):
    n_estimators = trial.suggest_int("n_estimators", 10, 200)
    max_depth = trial.suggest_int("max_depth", 2, 32, log=True)
    min_samples_split = trial.suggest_int("min_samples_split", 2, 10)

    model = RandomForestClassifier(
        class_weight='balanced',
        n_estimators=n_estimators,
        max_depth=max_depth,
        min_samples_split=min_samples_split,
        random_state=random_state
    )

    scores = cross_val_score(
```

```
        model,
        X=X_train,
        y=y_train,
        cv=cv,
        scoring="f1",
        n_jobs=-1,
    )
    f1 = scores.mean()
    return f1


study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=100)

top_5_f1_df = study.trials_dataframe().sort_values("value", ascending=
display(top_5_f1_df)
```

```
[I 2026-04-14 14:44:46,858] A new study created in memory with name: no
-name-bd465fdb-5a7e-40a5-8419-cd9e25ede6c2
[I 2026-04-14 14:44:48,804] Trial 0 finished with value: 0.668916432371
8346 and parameters: {'n_estimators': 35, 'max_depth': 5, 'min_samples_
split': 10}. Best is trial 0 with value: 0.6689164323718346.
[I 2026-04-14 14:44:50,992] Trial 1 finished with value: 0.667854481318
131 and parameters: {'n_estimators': 115, 'max_depth': 4, 'min_samples_
split': 4}. Best is trial 0 with value: 0.6689164323718346.
[I 2026-04-14 14:44:54,599] Trial 2 finished with value: 0.690574256352
4027 and parameters: {'n_estimators': 92, 'max_depth': 18, 'min_samples
_split': 3}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:44:56,789] Trial 3 finished with value: 0.665517972995
3378 and parameters: {'n_estimators': 95, 'max_depth': 5, 'min_samples_
split': 5}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:44:59,206] Trial 4 finished with value: 0.688823970778
2965 and parameters: {'n_estimators': 81, 'max_depth': 20, 'min_samples
_split': 6}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:00,215] Trial 5 finished with value: 0.668958088869
8118 and parameters: {'n_estimators': 81, 'max_depth': 5, 'min_samples_
split': 10}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:01,429] Trial 6 finished with value: 0.678945179261
0242 and parameters: {'n_estimators': 43, 'max_depth': 12, 'min_samples
_split': 2}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:02,135] Trial 7 finished with value: 0.669331172377
9061 and parameters: {'n_estimators': 57, 'max_depth': 4, 'min_samples_
split': 6}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:03,600] Trial 8 finished with value: 0.660867102479
0465 and parameters: {'n_estimators': 97, 'max_depth': 7, 'min_samples_
split': 7}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:08,973] Trial 9 finished with value: 0.657292373253
9245 and parameters: {'n_estimators': 173, 'max_depth': 26, 'min_sample
s_split': 3}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:09,952] Trial 10 finished with value: 0.64613355384
07518 and parameters: {'n_estimators': 148, 'max_depth': 2, 'min_sample
s_split': 2}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:10,513] Trial 11 finished with value: 0.65671323080
```

```
15359 and parameters: {'n_estimators': 11, 'max_depth': 29, 'min_sample
s_split': 8}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:13,953] Trial 12 finished with value: 0.68856036819
39719 and parameters: {'n_estimators': 130, 'max_depth': 17, 'min_sampl
es_split': 5}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:15,758] Trial 13 finished with value: 0.68525060202
31589 and parameters: {'n_estimators': 69, 'max_depth': 15, 'min_sample
s_split': 8}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:19,006] Trial 14 finished with value: 0.66213132542
61243 and parameters: {'n_estimators': 189, 'max_depth': 9, 'min_sample
s_split': 4}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:23,012] Trial 15 finished with value: 0.67523656364
19118 and parameters: {'n_estimators': 132, 'max_depth': 24, 'min_sampl
es_split': 6}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:25,058] Trial 16 finished with value: 0.68808229703
87469 and parameters: {'n_estimators': 80, 'max_depth': 15, 'min_sample
s_split': 3}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:28,475] Trial 17 finished with value: 0.68349842317
46521 and parameters: {'n_estimators': 114, 'max_depth': 22, 'min_sampl
es_split': 8}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:31,222] Trial 18 finished with value: 0.66452700839
73858 and parameters: {'n_estimators': 149, 'max_depth': 10, 'min_sampl
es_split': 4}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:32,623] Trial 19 finished with value: 0.68722144400
61214 and parameters: {'n_estimators': 44, 'max_depth': 19, 'min_sample
s_split': 6}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:32,917] Trial 20 finished with value: 0.29414649526
33757 and parameters: {'n_estimators': 13, 'max_depth': 2, 'min_samples
_split': 9}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:37,225] Trial 21 finished with value: 0.64003962388
78596 and parameters: {'n_estimators': 135, 'max_depth': 32, 'min_sampl
es_split': 5}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:40,001] Trial 22 finished with value: 0.68725714451
29589 and parameters: {'n_estimators': 107, 'max_depth': 15, 'min_sampl
es_split': 5}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:43,654] Trial 23 finished with value: 0.69029433851
80034 and parameters: {'n_estimators': 129, 'max_depth': 19, 'min_sampl
es_split': 3}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:45,312] Trial 24 finished with value: 0.66989496118
28121 and parameters: {'n_estimators': 79, 'max_depth': 11, 'min_sample
s_split': 3}. Best is trial 2 with value: 0.6905742563524027.
[I 2026-04-14 14:45:49,990] Trial 25 finished with value: 0.69173569198
27584 and parameters: {'n_estimators': 164, 'max_depth': 19, 'min_sampl
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:45:52,472] Trial 26 finished with value: 0.66286413491
69481 and parameters: {'n_estimators': 161, 'max_depth': 8, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:45:56,680] Trial 27 finished with value: 0.68519175915
44909 and parameters: {'n_estimators': 193, 'max_depth': 13, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:01,559] Trial 28 finished with value: 0.68866747487
56307 and parameters: {'n_estimators': 174, 'max_depth': 20, 'min_sampl
```

```
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:05,328] Trial 29 finished with value: 0.66442610614
67609 and parameters: {'n_estimators': 123, 'max_depth': 25, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:08,547] Trial 30 finished with value: 0.68320942976
4161 and parameters: {'n_estimators': 147, 'max_depth': 13, 'min_sample
s_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:11,157] Trial 31 finished with value: 0.68871763860
88202 and parameters: {'n_estimators': 94, 'max_depth': 18, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:13,107] Trial 32 finished with value: 0.68582775333
56544 and parameters: {'n_estimators': 62, 'max_depth': 20, 'min_sample
s_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:16,470] Trial 33 finished with value: 0.63856971718
87286 and parameters: {'n_estimators': 105, 'max_depth': 28, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:18,808] Trial 34 finished with value: 0.68703146897
02013 and parameters: {'n_estimators': 87, 'max_depth': 16, 'min_sample
s_split': 7}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:22,260] Trial 35 finished with value: 0.67996676396
35387 and parameters: {'n_estimators': 119, 'max_depth': 22, 'min_sampl
es_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:23,649] Trial 36 finished with value: 0.65225434692
75287 and parameters: {'n_estimators': 163, 'max_depth': 3, 'min_sample
s_split': 10}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:26,041] Trial 37 finished with value: 0.60722294990
08124 and parameters: {'n_estimators': 70, 'max_depth': 32, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:27,271] Trial 38 finished with value: 0.66352486056
29753 and parameters: {'n_estimators': 91, 'max_depth': 6, 'min_samples
_split': 7}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:28,520] Trial 39 finished with value: 0.68408217720
17329 and parameters: {'n_estimators': 48, 'max_depth': 12, 'min_sample
s_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:29,238] Trial 40 finished with value: 0.66600618931
81288 and parameters: {'n_estimators': 31, 'max_depth': 9, 'min_samples
_split': 6}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:31,973] Trial 41 finished with value: 0.68925275994
45065 and parameters: {'n_estimators': 100, 'max_depth': 18, 'min_sampl
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:35,206] Trial 42 finished with value: 0.67312845952
50001 and parameters: {'n_estimators': 106, 'max_depth': 23, 'min_sampl
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:38,944] Trial 43 finished with value: 0.68867857902
98859 and parameters: {'n_estimators': 141, 'max_depth': 17, 'min_sampl
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:41,258] Trial 44 finished with value: 0.68576094023
42912 and parameters: {'n_estimators': 98, 'max_depth': 14, 'min_sample
s_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:44,444] Trial 45 finished with value: 0.68823392508
39001 and parameters: {'n_estimators': 113, 'max_depth': 20, 'min_sampl
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
```

```
[I 2026-04-14 14:46:46,822] Trial 46 finished with value: 0.65628998109
53275 and parameters: {'n_estimators': 73, 'max_depth': 27, 'min_sample
s_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:50,608] Trial 47 finished with value: 0.67001223354
42368 and parameters: {'n_estimators': 200, 'max_depth': 11, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:54,098] Trial 48 finished with value: 0.68725014613
15146 and parameters: {'n_estimators': 127, 'max_depth': 18, 'min_sampl
es_split': 9}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:56,018] Trial 49 finished with value: 0.67838421390
23621 and parameters: {'n_estimators': 58, 'max_depth': 22, 'min_sample
s_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:46:57,780] Trial 50 finished with value: 0.65583310951
38449 and parameters: {'n_estimators': 178, 'max_depth': 4, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:00,829] Trial 51 finished with value: 0.68906518929
61971 and parameters: {'n_estimators': 97, 'max_depth': 17, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:03,875] Trial 52 finished with value: 0.68930755085
98343 and parameters: {'n_estimators': 99, 'max_depth': 16, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:06,987] Trial 53 finished with value: 0.68945870453
87677 and parameters: {'n_estimators': 101, 'max_depth': 16, 'min_sampl
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:09,578] Trial 54 finished with value: 0.68611585570
78625 and parameters: {'n_estimators': 88, 'max_depth': 14, 'min_sample
s_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:12,837] Trial 55 finished with value: 0.68884481077
84508 and parameters: {'n_estimators': 103, 'max_depth': 15, 'min_sampl
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:15,967] Trial 56 finished with value: 0.67878508491
98002 and parameters: {'n_estimators': 116, 'max_depth': 12, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:22,326] Trial 57 finished with value: 0.65536886146
71558 and parameters: {'n_estimators': 158, 'max_depth': 25, 'min_sampl
es_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:24,373] Trial 58 finished with value: 0.66408249234
68774 and parameters: {'n_estimators': 83, 'max_depth': 10, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:27,787] Trial 59 finished with value: 0.68943913832
0691 and parameters: {'n_estimators': 110, 'max_depth': 16, 'min_sample
s_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:30,028] Trial 60 finished with value: 0.66505033049
67457 and parameters: {'n_estimators': 127, 'max_depth': 6, 'min_sample
s_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:33,475] Trial 61 finished with value: 0.68942955736
52645 and parameters: {'n_estimators': 112, 'max_depth': 16, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:37,082] Trial 62 finished with value: 0.68942955736
52645 and parameters: {'n_estimators': 112, 'max_depth': 16, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:41,046] Trial 63 finished with value: 0.68504081840
```

```
87634 and parameters: {'n_estimators': 136, 'max_depth': 13, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:44,308] Trial 64 finished with value: 0.68863821966
1238 and parameters: {'n_estimators': 112, 'max_depth': 20, 'min_sample
s_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:47,127] Trial 65 finished with value: 0.68923043623
23524 and parameters: {'n_estimators': 109, 'max_depth': 16, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:50,557] Trial 66 finished with value: 0.68542659277
36384 and parameters: {'n_estimators': 119, 'max_depth': 21, 'min_sampl
es_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:53,030] Trial 67 finished with value: 0.67022316233
81121 and parameters: {'n_estimators': 121, 'max_depth': 11, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:47:57,618] Trial 68 finished with value: 0.67387819549
01397 and parameters: {'n_estimators': 152, 'max_depth': 24, 'min_sampl
es_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:00,124] Trial 69 finished with value: 0.62683635920
25077 and parameters: {'n_estimators': 75, 'max_depth': 30, 'min_sample
s_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:03,411] Trial 70 finished with value: 0.68565565507
0338 and parameters: {'n_estimators': 139, 'max_depth': 14, 'min_sample
s_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:05,985] Trial 71 finished with value: 0.68872348480
44812 and parameters: {'n_estimators': 92, 'max_depth': 18, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:08,853] Trial 72 finished with value: 0.68923043623
23524 and parameters: {'n_estimators': 109, 'max_depth': 16, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:09,781] Trial 73 finished with value: 0.65724267579
17427 and parameters: {'n_estimators': 102, 'max_depth': 3, 'min_sample
s_split': 2}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:13,562] Trial 74 finished with value: 0.69050499189
27524 and parameters: {'n_estimators': 128, 'max_depth': 19, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:17,783] Trial 75 finished with value: 0.68847623118
40558 and parameters: {'n_estimators': 131, 'max_depth': 20, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:20,619] Trial 76 finished with value: 0.68276177440
47016 and parameters: {'n_estimators': 125, 'max_depth': 13, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:25,325] Trial 77 finished with value: 0.68971263075
20576 and parameters: {'n_estimators': 169, 'max_depth': 19, 'min_sampl
es_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:30,985] Trial 78 finished with value: 0.66150759155
13654 and parameters: {'n_estimators': 185, 'max_depth': 26, 'min_sampl
es_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:35,379] Trial 79 finished with value: 0.67782445758
24919 and parameters: {'n_estimators': 144, 'max_depth': 23, 'min_sampl
es_split': 3}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:40,062] Trial 80 finished with value: 0.68957714306
37095 and parameters: {'n_estimators': 166, 'max_depth': 19, 'min_sampl
```

```
es_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:44,658] Trial 81 finished with value: 0.69017536533
09628 and parameters: {'n_estimators': 163, 'max_depth': 19, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:49,364] Trial 82 finished with value: 0.69024411903
12657 and parameters: {'n_estimators': 170, 'max_depth': 19, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:54,161] Trial 83 finished with value: 0.68461369012
04539 and parameters: {'n_estimators': 168, 'max_depth': 21, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:48:58,426] Trial 84 finished with value: 0.69008190758
39867 and parameters: {'n_estimators': 155, 'max_depth': 19, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:02,729] Trial 85 finished with value: 0.68794944873
66435 and parameters: {'n_estimators': 155, 'max_depth': 19, 'min_sampl
es_split': 6}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:07,349] Trial 86 finished with value: 0.69017795659
35231 and parameters: {'n_estimators': 165, 'max_depth': 19, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:12,652] Trial 87 finished with value: 0.67935343710
83089 and parameters: {'n_estimators': 173, 'max_depth': 23, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:18,403] Trial 88 finished with value: 0.64415208853
89173 and parameters: {'n_estimators': 182, 'max_depth': 30, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:22,766] Trial 89 finished with value: 0.68835743342
84932 and parameters: {'n_estimators': 161, 'max_depth': 18, 'min_sampl
es_split': 6}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:27,863] Trial 90 finished with value: 0.65308565004
30858 and parameters: {'n_estimators': 167, 'max_depth': 28, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:32,281] Trial 91 finished with value: 0.68551187868
99169 and parameters: {'n_estimators': 151, 'max_depth': 21, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:36,880] Trial 92 finished with value: 0.68875550307
22945 and parameters: {'n_estimators': 169, 'max_depth': 18, 'min_sampl
es_split': 4}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:42,633] Trial 93 finished with value: 0.69039064110
06462 and parameters: {'n_estimators': 177, 'max_depth': 19, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:48,801] Trial 94 finished with value: 0.67539799370
05778 and parameters: {'n_estimators': 177, 'max_depth': 24, 'min_sampl
es_split': 6}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:49:55,398] Trial 95 finished with value: 0.68226448393
63629 and parameters: {'n_estimators': 194, 'max_depth': 22, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:50:00,497] Trial 96 finished with value: 0.68989166247
73248 and parameters: {'n_estimators': 158, 'max_depth': 19, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:50:04,953] Trial 97 finished with value: 0.68633181690
38595 and parameters: {'n_estimators': 157, 'max_depth': 15, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
```

```
[I 2026-04-14 14:50:11,149] Trial 98 finished with value: 0.66601635615
80453 and parameters: {'n_estimators': 174, 'max_depth': 26, 'min_sampl
es_split': 6}. Best is trial 25 with value: 0.6917356919827584.
[I 2026-04-14 14:50:15,533] Trial 99 finished with value: 0.68844943786
36724 and parameters: {'n_estimators': 162, 'max_depth': 17, 'min_sampl
es_split': 5}. Best is trial 25 with value: 0.6917356919827584.
```

	number	value	datetime_start	datetime_complete	duration	pa
25	25	0.691736	2026-04-14 14:45:45.313099	2026-04-14 14:45:49.990891	0 days 00:00:04.677792	
2	2	0.690574	2026-04-14 14:44:50.993420	2026-04-14 14:44:54.599128	0 days 00:00:03.605708	
74	74	0.690505	2026-04-14 14:48:09.782988	2026-04-14 14:48:13.562496	0 days 00:00:03.779508	
93	93	0.690391	2026-04-14 14:49:36.881259	2026-04-14 14:49:42.633462	0 days 00:00:05.752203	
23	23	0.690294	2026-04-14 14:45:40.002773	2026-04-14 14:45:43.654819	0 days 00:00:03.652046	

In [54]:
```
# Train using optimal parameters
model = RandomForestClassifier(
    class_weight='balanced',
    n_estimators=top_5_f1_df.head(1)['params_n_estimators'].item(),
    max_depth=top_5_f1_df.head(1)['params_max_depth'].item(),
    min_samples_split=top_5_f1_df.head(1)['params_min_samples_split'].
    random_state=random_state
)
model.fit(X_train, y_train)
```

Out[54]:
```
▾ RandomForestClassifier
▶ Parameters
```

In [55]:
```
y_pred = model.predict(X_test)
```

In [56]:
```
print(classification_report(y_true=y_test, y_pred=y_pred))
```

```
              precision    recall  f1-score   support

           0       0.98      0.95      0.96     10355
           1       0.62      0.79      0.69      1035

    accuracy                           0.94     11390
   macro avg       0.80      0.87      0.83     11390
weighted avg       0.95      0.94      0.94     11390
```

In [57]:
```
f1_model_report = classification_report(y_true=y_test, y_pred=y_pred,
```

Compare Models

In [ ]:
```
# 2. Convert and transpose
recall_report_df = pd.DataFrame(recall_model_report_df).transpose()
f1_report_df = pd.DataFrame(f1_model_report).transpose()

# 3. Combine multiple reports (e.g., adding a 'Model' column for ident
combined_df = pd.concat([recall_report_df, f1_report_df], keys=['RF op
```

In [64]:
```
combined_df
```

Out[64]:

		precision	recall	f1-score	support
RF optimized for Recall	**0**	0.983590	0.931917	0.957056	10355.000000
	1	0.553515	0.844444	0.668707	1035.000000
	accuracy	0.923968	0.923968	0.923968	0.923968
	macro avg	0.768552	0.888181	0.812882	11390.000000
	weighted avg	0.944509	0.923968	0.930854	11390.000000
RF optimized for F1	**0**	0.978542	0.951231	0.964693	10355.000000
	1	0.618580	0.791304	0.694362	1035.000000
	accuracy	0.936699	0.936699	0.936699	0.936699
	macro avg	0.798561	0.871268	0.829528	11390.000000
	weighted avg	0.945832	0.936699	0.940128	11390.000000

In [65]:
```
model.feature_importances_
```

Out[65]:
```
array([6.62856785e-02, 6.05995047e-02, 9.03900169e-02, 1.01445840e-0
1,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+0
0,
       2.60059656e-04, 1.78374884e-03, 2.60533729e-04, 2.09601415e-0
4,
       1.49968033e-03, 2.93575262e-04, 4.40192161e-04, 3.55538205e-0
4,
       4.22583210e-03, 8.52128064e-04, 3.51972169e-04, 3.02110751e-0
4,
       9.16247728e-04, 3.06642333e-04, 3.21460516e-04, 1.69076506e-0
4,
       1.24906420e-03, 2.91538203e-04, 2.06885534e-04, 1.35531697e-0
3,
       2.81877643e-04, 3.16866904e-04, 3.73858263e-04, 4.76218125e-0
3,
       8.74444662e-04, 2.62872159e-04, 2.77801866e-04, 1.32006387e-0
3,
       2.70280515e-04, 5.92506764e-04, 3.84747851e-01, 1.02914776e-0
3,
       1.87227051e-02, 1.01112937e-01, 6.69776568e-02, 1.84186719e-0
2,
       9.61421113e-03, 3.74866401e-02, 1.81851808e-02])
```

In [73]:
```
pd.set_option('display.max_rows', None)
```

In [74]:
```
importances = model.feature_importances_
feature_names = df.drop(columns='Is Laundering').columns
feature_imp_df = pd.DataFrame({'Feature': feature_names, 'Gini Importa
    'Gini Importance', ascending=False)
feature_imp_df
```

Out[74]:

	Feature	Gini Importance
94	Payment Format_ACH	0.384748
3	Amount_Paid_USD	0.101446
97	Payment Format_Cheque	0.101113
2	Amount_Received_USD	0.090390
98	Payment Format_Credit Card	0.066978
0	Amount Received	0.066286
1	Amount Paid	0.060600
101	Account_Same	0.037487
96	Payment Format_Cash	0.018723
99	Payment Format_Reinvestment	0.018419
102	Bank_Same	0.018185
100	Payment Format_Wire	0.009614
87	Payment Currency_Saudi Riyal	0.004762
72	Receiving Currency_Saudi Riyal	0.004226
65	Receiving Currency_Bitcoin	0.001784
68	Receiving Currency_Euro	0.001500
83	Payment Currency_Euro	0.001355
91	Payment Currency_US Dollar	0.001320
80	Payment Currency_Bitcoin	0.001249
95	Payment Format_Bitcoin	0.001029
76	Receiving Currency_US Dollar	0.000916
88	Payment Currency_Shekel	0.000874
73	Receiving Currency_Shekel	0.000852
93	Payment Currency_Yuan	0.000593
70	Receiving Currency_Ruble	0.000440

86	Payment Currency_Rupee	0.000374
71	Receiving Currency_Rupee	0.000356
74	Receiving Currency_Swiss Franc	0.000352
78	Receiving Currency_Yuan	0.000321
85	Payment Currency_Ruble	0.000317
77	Receiving Currency_Yen	0.000307
75	Receiving Currency_UK Pound	0.000302
69	Receiving Currency_Mexican Peso	0.000294
81	Payment Currency_Brazil Real	0.000292
84	Payment Currency_Mexican Peso	0.000282
90	Payment Currency_UK Pound	0.000278
92	Payment Currency_Yen	0.000270
89	Payment Currency_Swiss Franc	0.000263
66	Receiving Currency_Brazil Real	0.000261
64	Receiving Currency_Australian Dollar	0.000260
67	Receiving Currency_Canadian Dollar	0.000210
82	Payment Currency_Canadian Dollar	0.000207
79	Payment Currency_Australian Dollar	0.000169
38	UK Pound_Amount_Received_Mean	0.000000
37	UK Pound_Amount_Paid_Median	0.000000
36	UK Pound_Amount_Paid_Mean	0.000000
31	Yen_Amount_Received_Median	0.000000
30	Yen_Amount_Received_Mean	0.000000
29	Yen_Amount_Paid_Median	0.000000
28	Yen_Amount_Paid_Mean	0.000000
27	Rupee_Amount_Received_Median	0.000000
26	Rupee_Amount_Received_Mean	0.000000
25	Rupee_Amount_Paid_Median	0.000000
24	Rupee_Amount_Paid_Mean	0.000000
23	Yuan_Amount_Received_Median	0.000000
22	Yuan_Amount_Received_Mean	0.000000

21	Yuan_Amount_Paid_Median	0.000000
20	Yuan_Amount_Paid_Mean	0.000000
19	Australian Dollar_Amount_Received_Median	0.000000
18	Australian Dollar_Amount_Received_Mean	0.000000
17	Australian Dollar_Amount_Paid_Median	0.000000
16	Australian Dollar_Amount_Paid_Mean	0.000000
15	Euro_Amount_Received_Median	0.000000
14	Euro_Amount_Received_Mean	0.000000
13	Euro_Amount_Paid_Median	0.000000
12	Euro_Amount_Paid_Mean	0.000000
11	Bitcoin_Amount_Received_Median	0.000000
10	Bitcoin_Amount_Received_Mean	0.000000
9	Bitcoin_Amount_Paid_Median	0.000000
8	Bitcoin_Amount_Paid_Mean	0.000000
7	US Dollar_Amount_Received_Median	0.000000
6	US Dollar_Amount_Received_Mean	0.000000
5	US Dollar_Amount_Paid_Median	0.000000
4	US Dollar_Amount_Paid_Mean	0.000000
35	Mexican Peso_Amount_Received_Median	0.000000
34	Mexican Peso_Amount_Received_Mean	0.000000
33	Mexican Peso_Amount_Paid_Median	0.000000
32	Mexican Peso_Amount_Paid_Mean	0.000000
63	Shekel_Amount_Received_Median	0.000000
55	Brazil Real_Amount_Received_Median	0.000000
56	Saudi Riyal_Amount_Paid_Mean	0.000000
57	Saudi Riyal_Amount_Paid_Median	0.000000
58	Saudi Riyal_Amount_Received_Mean	0.000000
59	Saudi Riyal_Amount_Received_Median	0.000000
60	Shekel_Amount_Paid_Mean	0.000000
61	Shekel_Amount_Paid_Median	0.000000
62	Shekel_Amount_Received_Mean	0.000000

46	Canadian Dollar_Amount_Received_Mean	0.000000
45	Canadian Dollar_Amount_Paid_Median	0.000000
44	Canadian Dollar_Amount_Paid_Mean	0.000000
43	Ruble_Amount_Received_Median	0.000000
42	Ruble_Amount_Received_Mean	0.000000
41	Ruble_Amount_Paid_Median	0.000000
40	Ruble_Amount_Paid_Mean	0.000000
47	Canadian Dollar_Amount_Received_Median	0.000000
39	UK Pound_Amount_Received_Median	0.000000
48	Swiss Franc_Amount_Paid_Mean	0.000000
49	Swiss Franc_Amount_Paid_Median	0.000000
50	Swiss Franc_Amount_Received_Mean	0.000000
51	Swiss Franc_Amount_Received_Median	0.000000
52	Brazil Real_Amount_Paid_Mean	0.000000
53	Brazil Real_Amount_Paid_Median	0.000000
54	Brazil Real_Amount_Received_Mean	0.000000

In [ ]:

4. Storytelling With Data plot

Reproduce any graph of your choice in chapter seven (p. 165-185) of the Storytelling With Data book as best you can. You do not have to get the exact data values right, just the overall look and feel.

In [ ]:

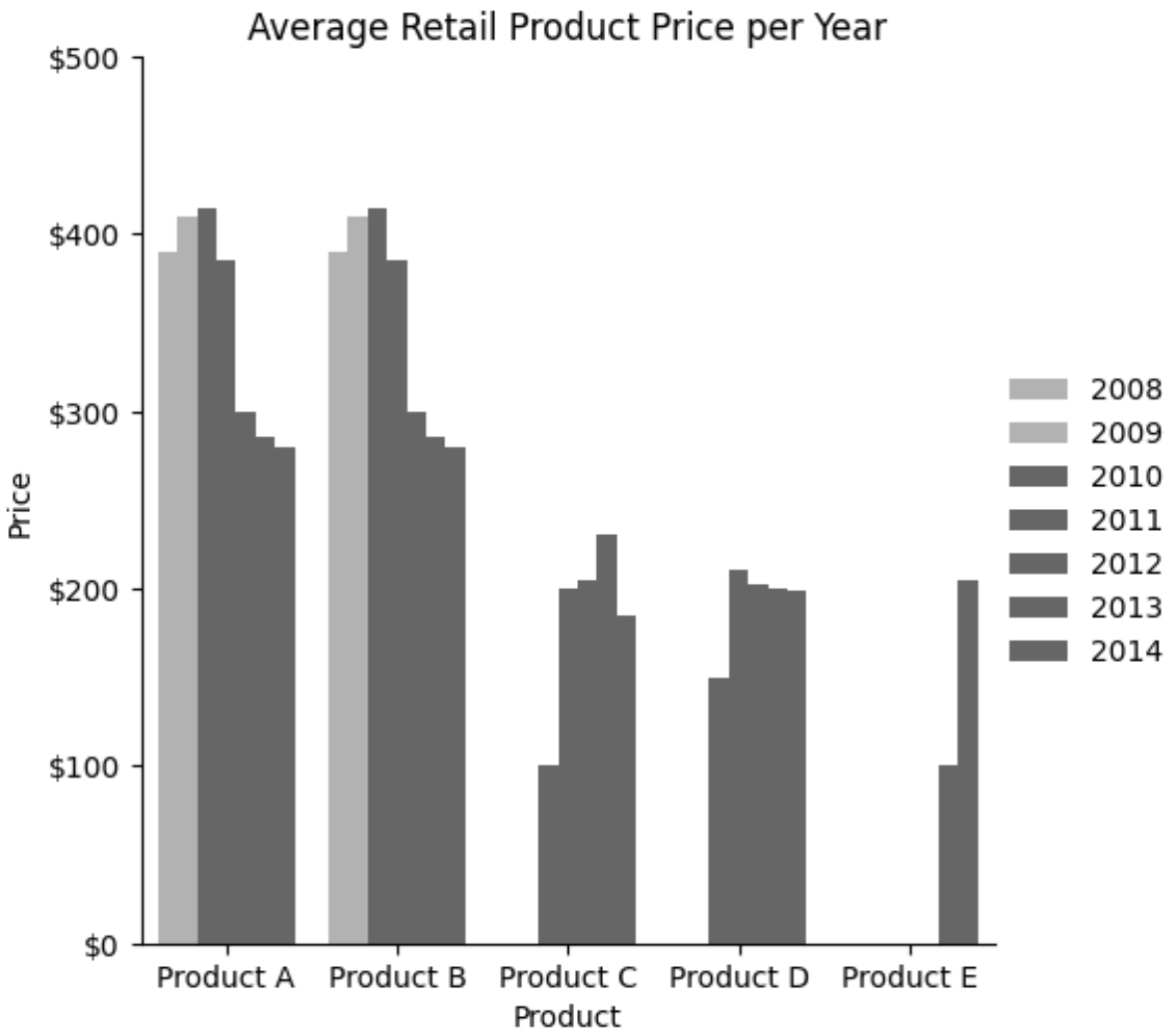
```
# Figure 8.3 (p. 191)
import seaborn as sns
import matplotlib.ticker as tkr

products = [
    'Product A',
    'Product A',
    'Product A',
    'Product A',
    'Product A',
    'Product A',
    'Product A',
    'Product B',
```

```
    'Product B',
    'Product B',
    'Product B',
    'Product B',
    'Product B',
    'Product B',
    'Product C',
    'Product C',
    'Product C',
    'Product C',
    'Product C',
    'Product C',
    'Product C',
    'Product D',
    'Product D',
    'Product D',
    'Product D',
    'Product D',
    'Product D',
    'Product D',
    'Product E',
    'Product E',
    'Product E',
    'Product E',
    'Product E',
    'Product E',
    'Product E',
]
years = [
    # A
    '2008',
    '2009',
    '2010',
    '2011',
    '2012',
    '2013',
    '2014',
    # B
    '2008',
    '2009',
    '2010',
    '2011',
    '2012',
    '2013',
    '2014',
    # C
    '2008',
    '2009',
    '2010',
    '2011',
    '2012',
    '2013',
```

```
    '2014',
    # D
    '2008',
    '2009',
    '2010',
    '2011',
    '2012',
    '2013',
    '2014',
    # E
    '2008',
    '2009',
    '2010',
    '2011',
    '2012',
    '2013',
    '2014',
]
prices = [
    # A
    390,
    410,
    415,
    385,
    300,
    285,
    280,
    # B
    390,
    410,
    415,
    385,
    300,
    285,
    280,
    # C
    0,
    0,
    100,
    200,
    205,
    230,
    185,
    # D
    0,
    0,
    150,
    210,
    202,
    200,
    199,
    # E
```

```
    0,
    0,
    0,
    0,
    0,
    100,
    205,
]
df = pd.DataFrame({
    'Product': products,
    'Year': years,
    'Price': prices
})

# Draw a nested barplot by species and sex
colors = ["gray", "gray", "black", "black", "black", "black", "black"]

g = sns.catplot(
    data=df, kind="bar",
    x="Product", y="Price", hue="Year",
    palette=colors, alpha=.6,
)
# Apply dollar sign formatting
fmt = '${x:,.0f}' # Adds $ and comma separators
tick = tkr.StrMethodFormatter(fmt)
for ax in g.axes.flat:
    ax.yaxis.set_major_formatter(tick)

plt.ylim(None, 500) # Sets y-max to 100

g.legend.set_title("")
plt.title("Average Retail Product Price per Year")
```

Out[ ]: Text(0.5, 1.0, 'Average Retail Product Price per Year')


Average Retail Product Price per Year
$500
$400
$300
$200
$100
$0
Price
Product A
Product B
Product C
Product D
Product E
Product
2008
2009
2010
2011
2012
2013
2014

Week 12 - Milestone Two week

1. Lesson: no lesson this week. Milestone Two is due!

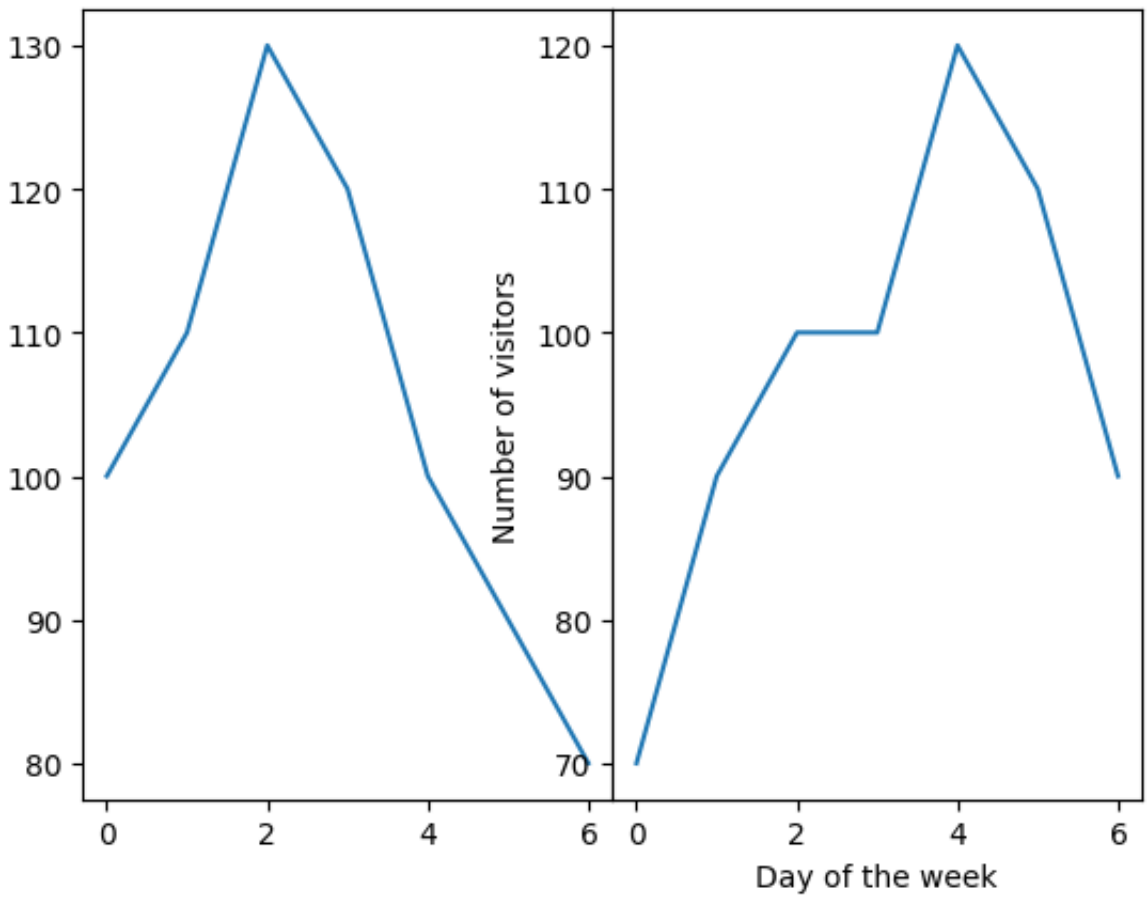
2. Weekly graph question

Suppose you plot the number visitors to a museum over the seven days of a week; then again the following week. Can you suggest the advantages and disadvantages of the following two ways of presenting the data?

In [1]:
```
import numpy as np
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt
```

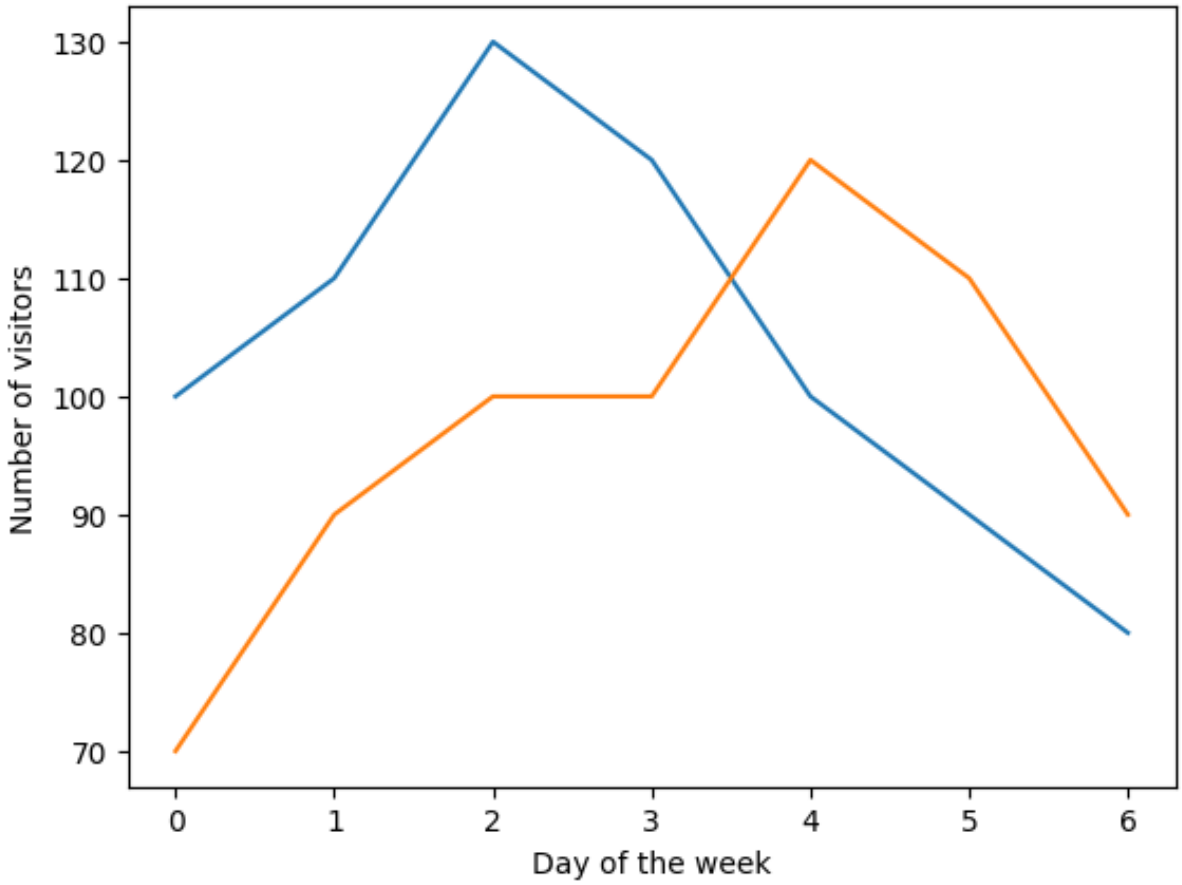
In [17]:
```
np.random.seed(0)
time_series_0 = np.array([100, 110, 130, 120, 100, 90, 80])
time_series_1 = np.array([70, 90, 100, 100, 120, 110, 90])
fig, axs = plt.subplots(1, 2)
axs[0].plot(time_series_0)
axs[1].plot(time_series_1)
plt.subplots_adjust(wspace = 0)
plt.xlabel("Day of the week")
plt.ylabel("Number of visitors")
```

Out[17]:
```
Text(0, 0.5, 'Number of visitors')
```



In [18]:
```
plt.plot(time_series_0)
plt.plot(time_series_1)
plt.xlabel("Day of the week")
plt.ylabel("Number of visitors")
```

Out[18]: Text(0, 0.5, 'Number of visitors')



The top plot has some disadvantages. First, the scale of the y axis on the side by side plots are different, making it difficult to compare the data. Additionally, since the lines do not overlap like in the second plot, it is difficult to compare each day between the two weeks. Next, you cannot tell which plot corresponds to which week. Better labeling should be added to indicate that. Finally, the Day of the week should be changed to text to make it more clear.

The second plot does a better job at displaying the data in a way that can be compared. One improvement that is still needed is that a legend should be added. In the current plot, you cannot tell which line correpsonds to which week. Additionally, the Day of the week should be changed to text such as 0 -> Monday which would make it more clear.

3. Working on your datasets

This week, you can work on Milestone Two rather than having a particular exercise about working on your datasets

4. Storytelling With Data plot

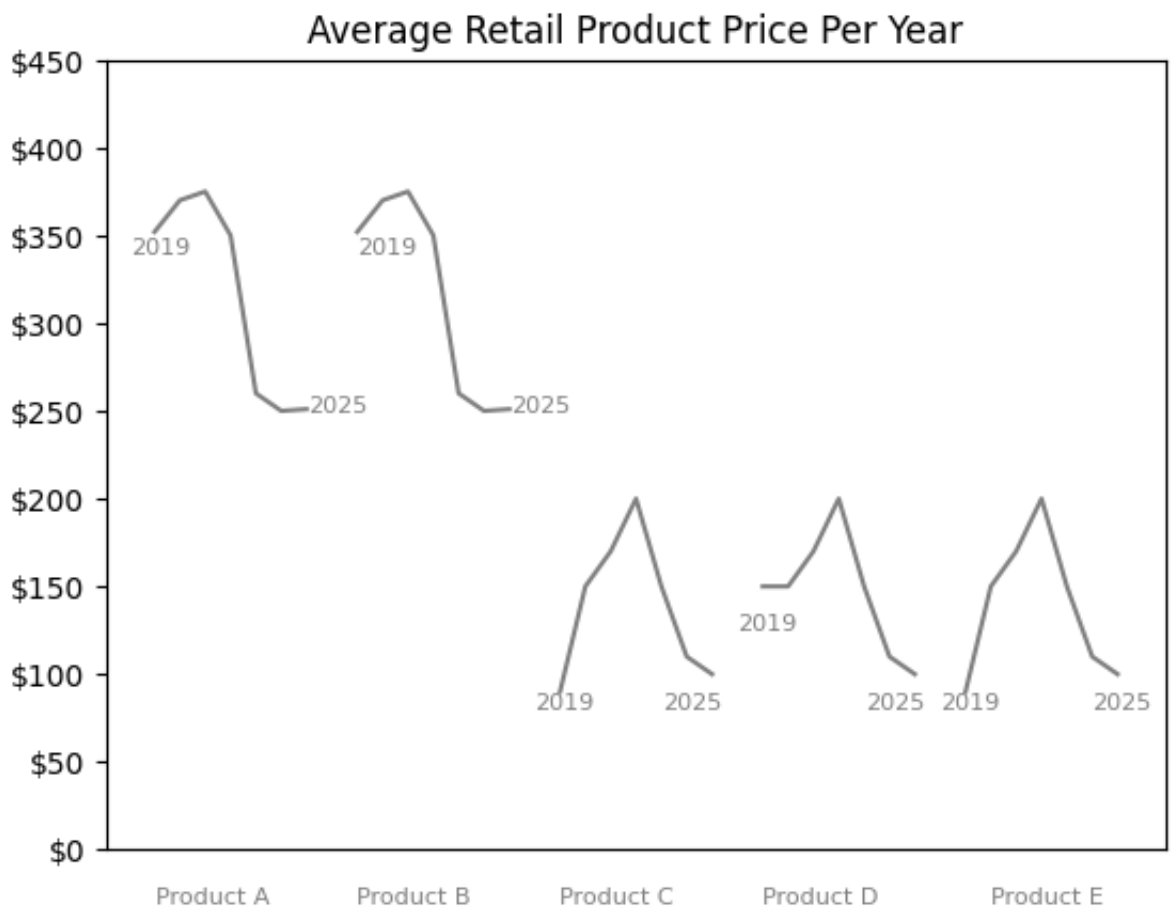
Reproduce any graph of your choice in p. 187-205 of the Storytelling With Data book as best you can. (Chapter Eight). You do not have to get the exact data values right, just the overall look and feel.

In [14]:
```
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
```

In [ ]:
```
# Chapter 8 fig 4
x1 = [0,1,2,3,4,5,6]
y1 = [352, 370, 375, 350, 260, 250, 251]

x2 = [8,9,10,11,12,13,14]
y2 = [352, 370, 375, 350, 260, 250, 251]

x3 = [16,17,18,19,20,21,22]
y3 = [90, 150, 170, 200, 150, 110, 100]

x4 = [24,25,26,27,28,29,30]
y4 = [150, 150, 170, 200, 150, 110, 100]

x5 = [32,33,34,35,36,37,38]
y5 = [90, 150, 170, 200, 150, 110, 100]

fig, ax = plt.subplots()

ax.plot(x1, y1, color='gray')
ax.plot(x2, y2, color='gray')
ax.plot(x3, y3, color='gray')
ax.plot(x4, y4, color='gray')
ax.plot(x5, y5, color='gray')

plt.ylim(0, 450)
plt.xticks([])

plt.text(-1, 340, '2019', fontsize=8, color='gray')
plt.text(6, 250, '2025', fontsize=8, color='gray')

plt.text(8, 340, '2019', fontsize=8, color='gray')
plt.text(14, 250, '2025', fontsize=8, color='gray')

plt.text(15, 80, '2019', fontsize=8, color='gray')
plt.text(20, 80, '2025', fontsize=8, color='gray')

plt.text(23, 125, '2019', fontsize=8, color='gray')
plt.text(28, 80, '2025', fontsize=8, color='gray')

plt.text(31, 80, '2019', fontsize=8, color='gray')
plt.text(37, 80, '2025', fontsize=8, color='gray')

plt.text(0, -30, 'Product A', fontsize=8, color='gray')
plt.text(8, -30, 'Product B', fontsize=8, color='gray')
```

```
plt.text(16, -30, 'Product C', fontsize=8, color='gray')
plt.text(24, -30, 'Product D', fontsize=8, color='gray')
plt.text(33, -30, 'Product E', fontsize=8, color='gray')


fmt = '${x:,.0f}'
tick = mtick.StrMethodFormatter(fmt)
ax.yaxis.set_major_formatter(tick)

plt.title("Average Retail Product Price Per Year")
```

Out[ ]: Text(0.5, 1.0, 'Average Retail Product Price Per Year')



In [ ]:

In [ ]:

In [ ]: